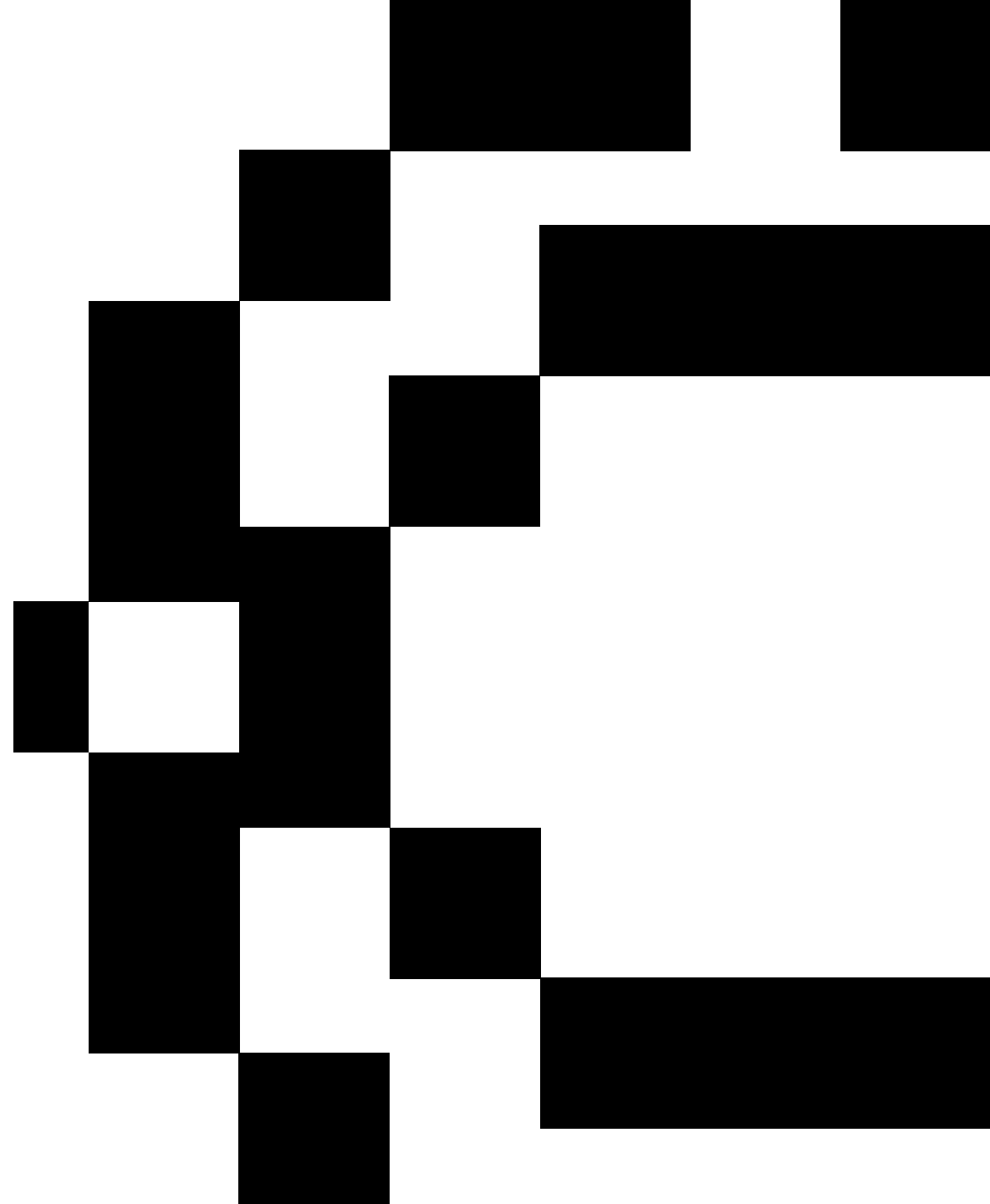


03: Model Requirements

Nuno Rosa
nagostinho@novaims.unl.pt





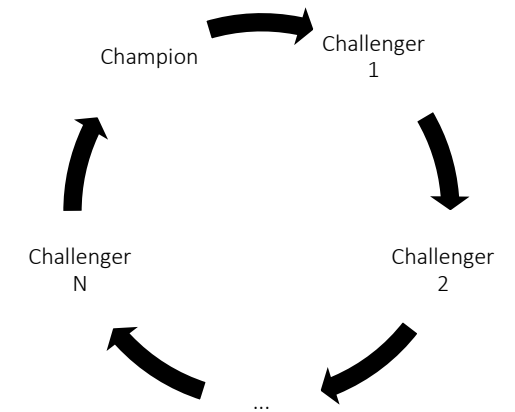
Last week continuation and recap

02: Model Deployment and Production

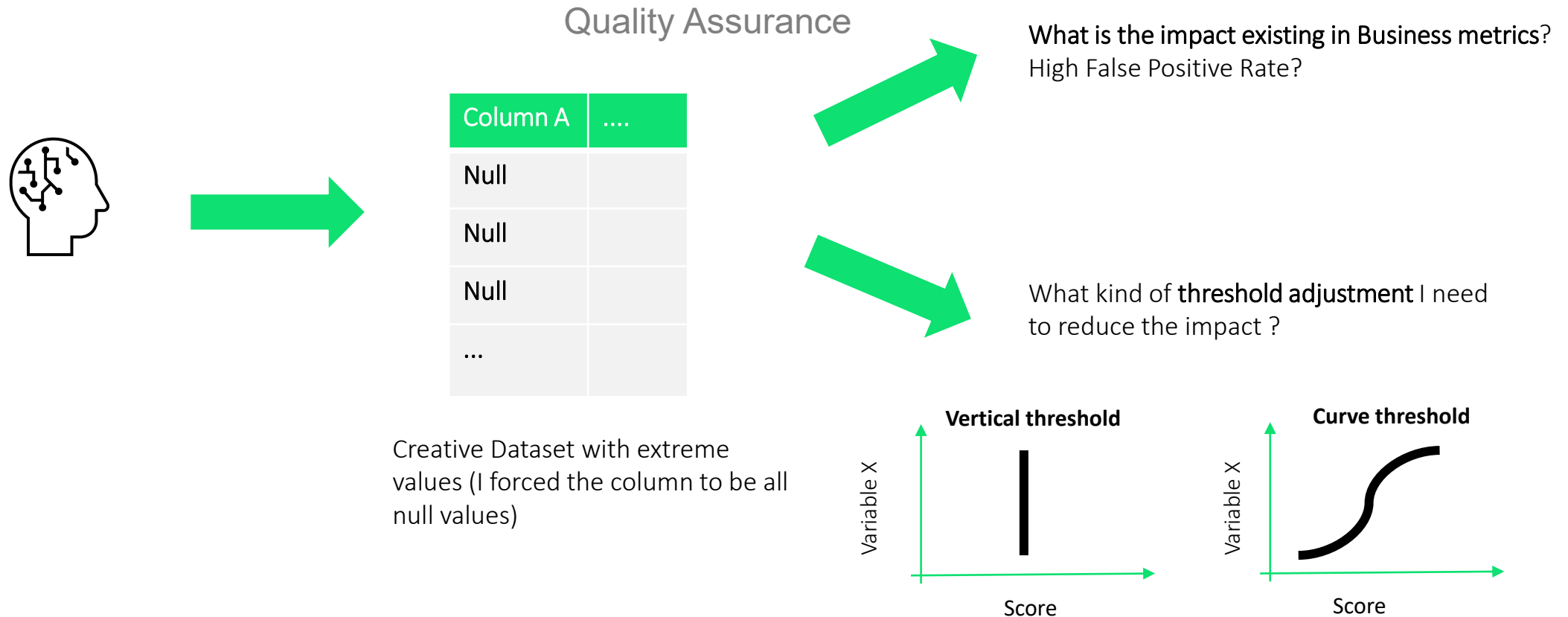
Quality Assurance

- Unlikely to software engineering, quality assurance methodologies in MLOps is still giving the first steps, and it is too much dependent on the degree of maturity of the data organization and appended teams.
- The entire process of data collection, data ingestion, data transformation, feature engineering model training and evaluation metrics should all be documented. It will be easier to validate every step of the process against organizational guidelines.
- Don't be spared about having assertions over your deploy pipeline. Create useful metrics as quality requirements for deployments.

Quality metrics	Model performance on production data (precision, accuracy, recall, e.g. an assertion on if the model accuracy falls below 90%). Metrics for computational performance (average latency, e.g. an assertion if the 5% of score events take more than x milliseconds).
Creative Scenarios	A feature (or more) that only has missing values. How does the performance changes? The distribution of several features changes (one can force some events to have extreme values). Can the process be used for 3x times the quantity of the expected data.
Subpopulation analysis and model fairness	Overall metrics of the model are very good. But splitting the data into subpopulations it seems that there exists a concentration of false positives in a specific segment.



02: Model Deployment and Production



Even I have a strong layer of data unit tests, there will be some time to react. Meanwhile I need to have some mitigation measures.

02: Model Deployment and Production

Real-world examples



Ecommerce card fraud detection:

- Clients without recent history of usage have a much more higher rate of rejected first transaction (this can affect user experience).
- Trained dataset without any period of holidays peak season -> high false positive rate on transactions for travel agencies, flight companies...
- Trained dataset sampled with the majority of fraud highly correlated with the age and education of victims – blocking certain groups can affect bank reputation.



Healthcare:

- Prediction of patients that would likely need extra medical care.
- There was a feature highly correlated to race was, which was healthcare cost history.
- The rationale was that cost summarizes how many healthcare needs a particular person has.
- For various reasons, black patients incurred lower healthcare costs than white patients with the same conditions on average.



Vehicle stock and sales recommendation:

- Model developed for a specific market.
- When the model was world-wide scaled, could not handle efficiently the new amount of data.

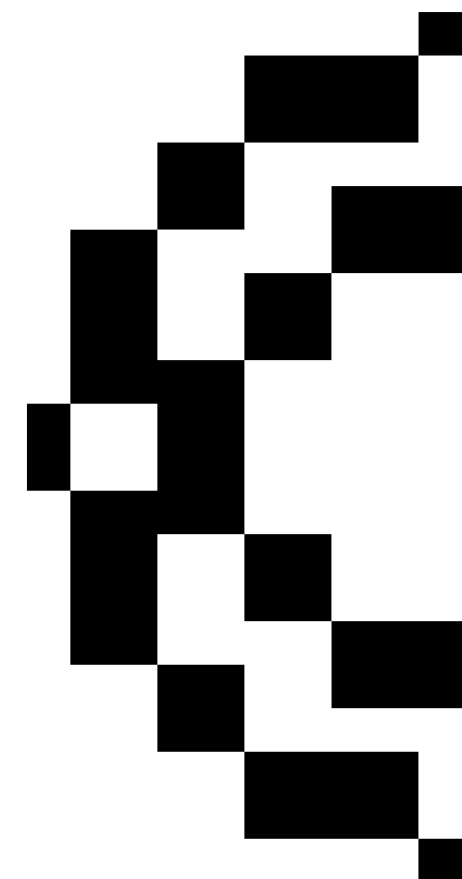
02: Model Deployment and Production

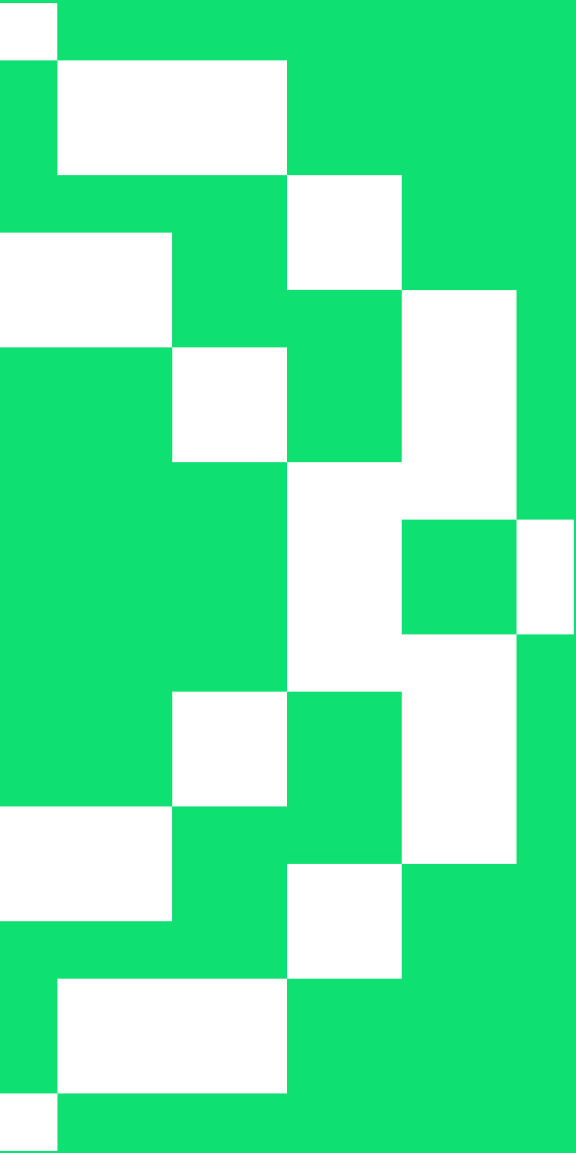
Risk

- Production data changes for some reason (world war, seasonality, bug on a system). The value of a feature at inference time is out of the bounds. Should we have process where the system can trigger appropriate measures, such as rejecting the sample or dispatching a warning message?
- In a classification problem, each event is scored and a threshold is defined for the final decision. Facing any seasonality event, one can have some dynamic threshold that will be responsible for accepting an inference output, or anomaly detection algorithm to find the events that are outside of the application domain.
- Model with a high cardinality of feature will be more sensitive to these issues: the number of combinations is exponential, so the performance could be unpredictable.
- Is the application domain of a new model already occupied by another model or heuristic rules? Should the outcome be combined? What strategy should be done to switch models, if it's the case?

Security

- There should be several access layers to the training dataset and model output. Models can leak data, for example, a model predicts how much someone is paid using the nearest neighbor algorithm. If one knows the zip code, age, and profession of a certain person registered on the service, the leak will happen...
- Connections to the external environment of the organization. If we need enrich the data while the model acting on real time, there is a chance an attacker be able to change the training data or to introduce small perturbations on the production data.

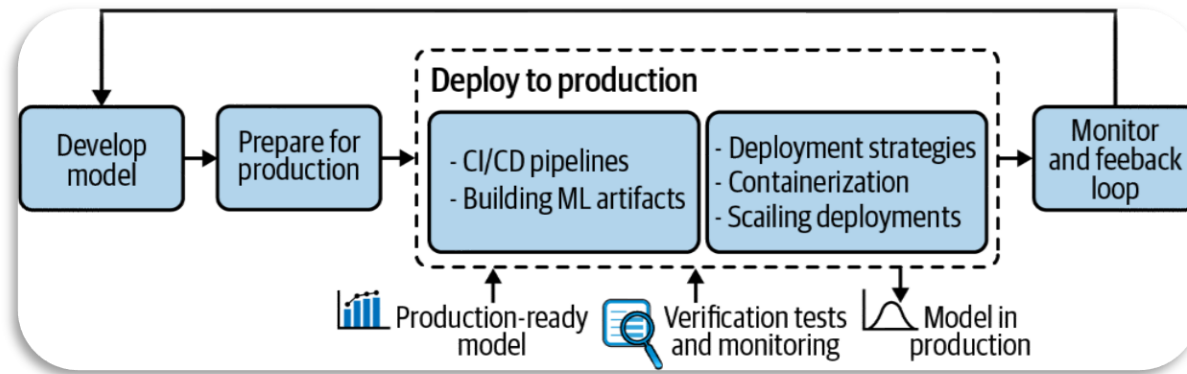




Model Deployment

Production Requirements

03: Model Deployment Requirements



- In this stage we need to have a ML artifact: **a testable and deployable bundle of the project in the context of the CI/CD**. This artifact can only be built after **we have a modular code and data centralized in a repository** with several elements to be bundled into an artifact:
 1. Code for preprocessing, model, hyperparameters.
 2. Environment with the libraries, specific versions, environment variables...
 3. Training, validation and testing scenarios data.
- The artifact goes through a testing and standard process, and it is made available for deployment.

It is crucial to have several validation steps to ensure the model fulfils all production requirements: from software to computation performance.

03: Model Deployment Requirements

The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction

Eric Breck, Shanqing Cai, Eric Nielsen, Michael Salib, D. Sculley
Google, Inc.
ebreck, cais, nielsene, msalib, dsculley@google.com

Abstract—Creating reliable, production-level machine learning systems brings on a host of concerns not found in small toy examples or even large offline research experiments. Testing and monitoring are key considerations for ensuring the production-readiness of an ML system, and for reducing technical debt of ML systems. But it can be difficult to formulate specific tests, given that the actual prediction behavior of any given model is difficult to specify *a priori*. In this paper, we present 28 specific tests and monitoring needs, drawn from experience with a wide range of production ML systems to help quantify these issues and present an easy to follow road-map to improve production readiness and pay down ML technical debt.

Keywords—Machine Learning, Testing, Monitoring, Reliability, Best Practices, Technical Debt

I. INTRODUCTION

As machine learning (ML) systems continue to take on ever more central roles in real-world production settings, the issue of *ML reliability* has become increasingly critical. ML reliability involves a host of issues not found in small toy examples or even large offline experiments, which can lead to surprisingly large amounts of technical debt [1]. Testing and monitoring are important strategies for improving reliability, reducing technical debt, and lowering long-term maintenance cost. However, as suggested by Figure 1, ML system testing is also more complex a challenge than testing manually coded systems, due to the fact that

may find difficult. Note that this rubric focuses on issues specific to ML systems, and so does not include generic software engineering best practices such as ensuring good unit test coverage and a well-defined binary release process. Such strategies remain necessary as well. We do call out a few specific areas for unit or integration tests that have unique ML-related behavior.

How to read the tests: Each test is written as an assertion; our recommendation is to test that the assertion is true, the more frequently the better, and to fix the system if the assertion is not true.

Doesn't this all go without saying?: Before we enumerate our suggested tests, we should address one objection the reader may have – obviously one should write tests for an engineering project! While this is true in principle, in a survey of several dozen teams at Google, *none* of these tests was implemented by more than 80% of teams (though, even in an engineering culture valuing rigorous testing, many of these ML-centric tests are non-obvious). Conversely, most tests had a nonzero score for at least half of the teams surveyed; our tests do represent practices that teams find to be worth doing.

In this paper, we are largely concerned with supervised ML systems that are trained continuously online and perform rapid, low-latency inference on a server. Features are often derived from large amounts of data such as streaming logs

<https://research.google/pubs/the-ml-test-score-a-rubric-for-ml-production-readiness-and-technical-debt-reduction/>

03: Model Deployment Requirements

DATA TESTS

1	Feature expectations are captured in a schema.
2	All features are beneficial.
3	No feature's cost is too much.
4	Features adhere to meta-level requirements.
5	The data pipeline has appropriate privacy controls.
6	New features can be added quickly.
7	All input feature code is tested.

ML INFRASTRUCTURE TESTS

1	Training is reproducible.
2	Model specs are unit tested.
3	The ML pipeline is Integration tested.
4	Model quality is validated before serving.
5	The model is debuggable.
6	Models are canaried before serving.
7	Serving models can be rolled back.

MODEL TESTS

1	Model specs are reviewed and submitted.
2	Offline and online metrics correlate.
3	All hyperparameters have been tuned.
4	The impact of model staleness is known.
5	A simpler model is not better.
6	Model quality is sufficient on important data slices.
7	The model is tested for considerations of inclusion.

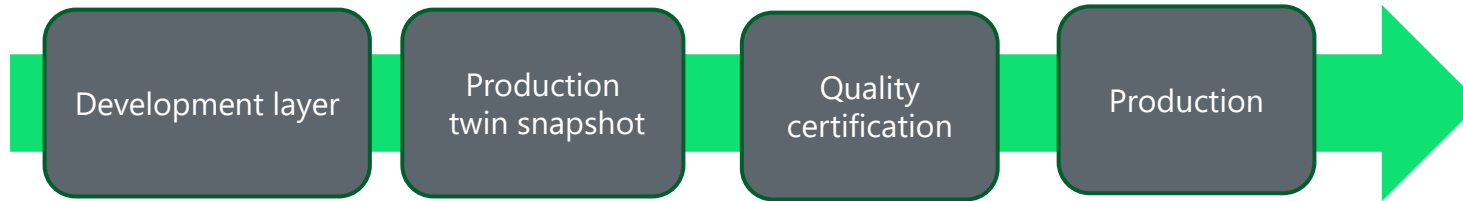
MONITORING TESTS

1	Dependency changes result in notification.
2	Data invariants hold for inputs.
3	Training and serving are not skewed.
4	Models are not too stale.
5	Models are numerically stable.
6	Computing performance has not regressed.
7	Prediction quality has not regressed.

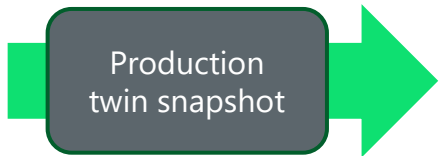
<https://research.google/pubs/the-ml-test-score-a-rubric-for-ml-production-readiness-and-technical-debt-reduction/>

03: Model Deployment Requirements

Preproduction environments



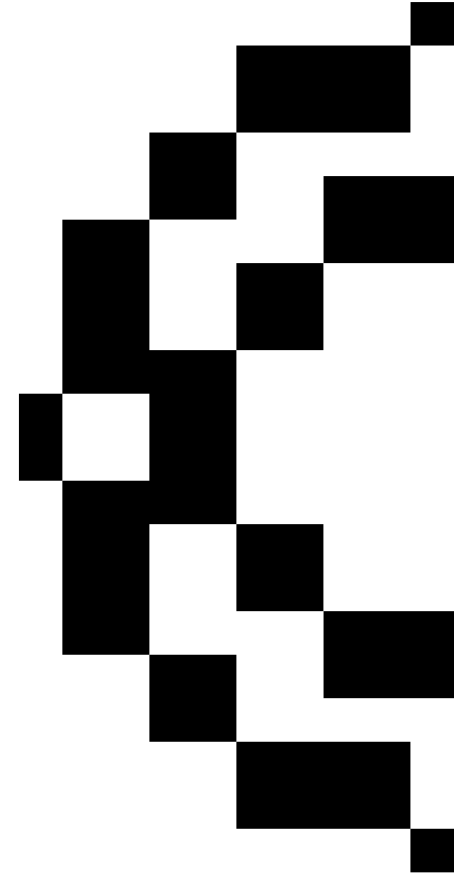
- Testing the **model combability with production requirements** (version compatibility, possible crashes).



- Check if the **features for training and serving are the same**.
- Check model scoring values vs previous obtain in the model development environment.



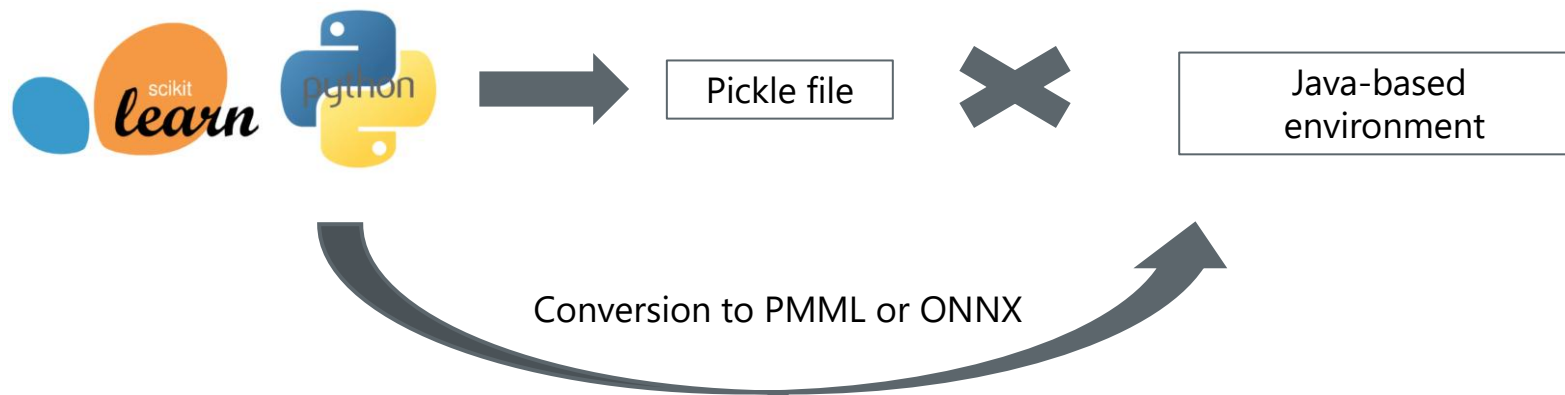
- Real time injection of a sample of events.
- Test the **latency of the model** accordingly to the service standards.
- Python based serving systems can have more latency than its equivalent in C++ .
- Check **score distribution and bias** on sample events.



03: Model Deployment Requirements

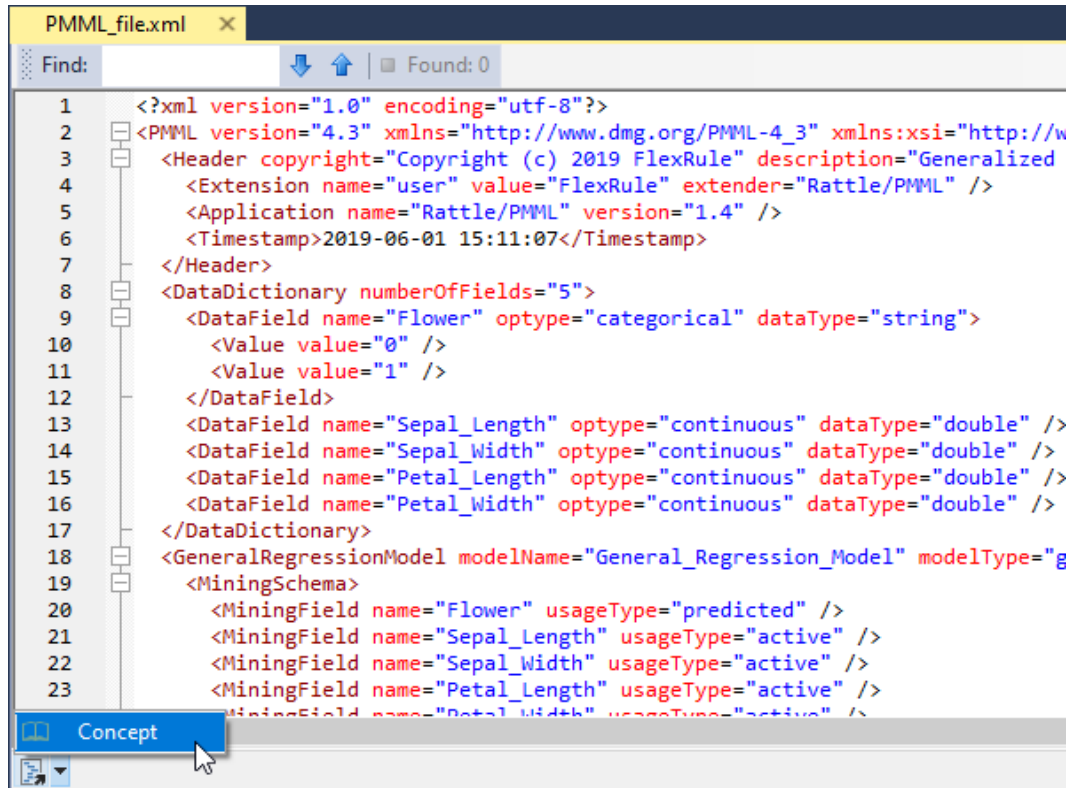
Real example

When developing a model, we need to consider earlier what format is required in production environment, since it may have additional impact on the development of additional transformations, e.g.:



03: Model Deployment Requirements

Real example



```
1 <?xml version="1.0" encoding="utf-8"?>
2 <PMML version="4.3" xmlns="http://www.dmg.org/PMML-4_3" xmlns:xsi="http://www.dmg.org/PMML-4_3.xsd" />
3 <Header copyright="Copyright (c) 2019 FlexRule" description="Generalized" />
4 <Extension name="user" value="FlexRule" extender="Rattle/PMML" />
5 <Application name="Rattle/PMML" version="1.4" />
6 <Timestamp>2019-06-01 15:11:07</Timestamp>
7 </Header>
8 <DataDictionary numberOfFields="5">
9 <DataField name="Flower" optype="categorical" dataType="string">
10 <Value value="0" />
11 <Value value="1" />
12 </DataField>
13 <DataField name="Sepal_Length" optype="continuous" dataType="double" />
14 <DataField name="Sepal_Width" optype="continuous" dataType="double" />
15 <DataField name="Petal_Length" optype="continuous" dataType="double" />
16 <DataField name="Petal_Width" optype="continuous" dataType="double" />
17 </DataDictionary>
18 <GeneralRegressionModel modelName="General_Regression_Model" modelType="g" />
19 <MiningSchema>
20 <MiningField name="Flower" usageType="predicted" />
21 <MiningField name="Sepal_Length" usageType="active" />
22 <MiningField name="Sepal_Width" usageType="active" />
23 <MiningField name="Petal_Length" usageType="active" />
24 <MiningField name="Petal_Width" usageType="active" />
25 </MiningSchema>
26 </GeneralRegressionModel>
27 </PMML>
```

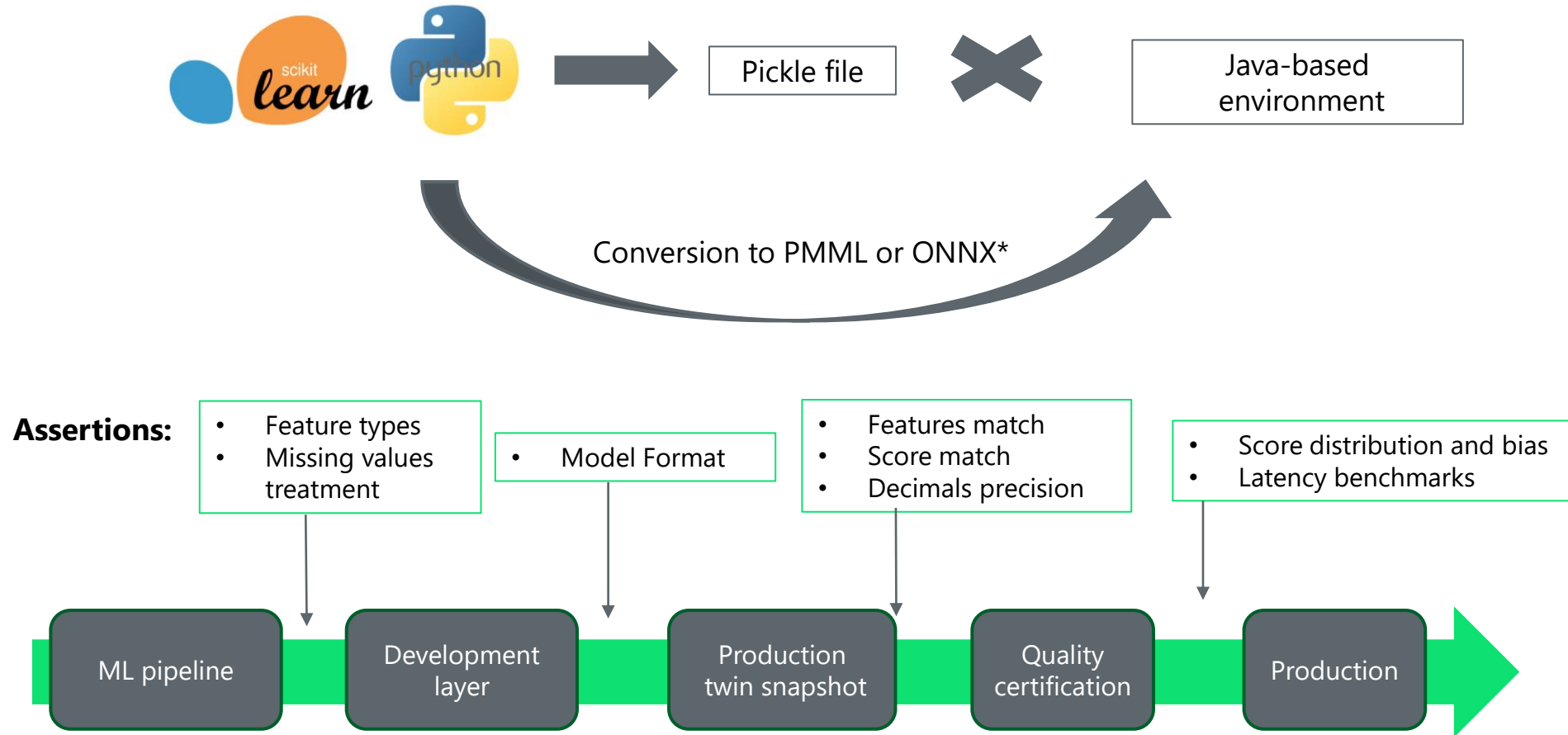
- A PMML file, is simply a based XML file.
- The PMML specification defines producer and consumer conformance levels in a software implementation to ensure that PMML models are created and integrated reliably.
- It contains several fields that provide the relevant information for production environment:

Field	Description
Header	Version and timestamp Model development environment information
Data Dictionary	Definition o variable types, valid, invalid and missing values
Data Transformation	Normalization, mapping and discretization Data aggregation and function calls
Model	Description and model specific atributes
Mining Schema	Usage type, outlier and missing value treatment and replacement
Targets	Score post-processing - scaling Definition of model architecture/ parameters

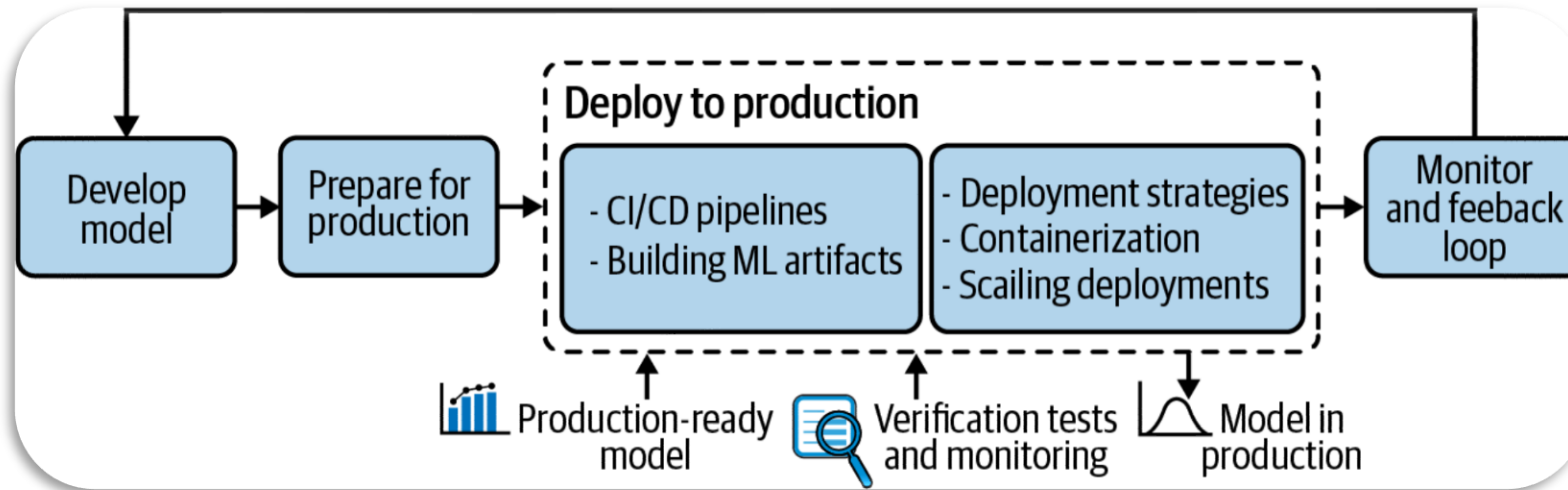
03: Model Deployment Requirements

Real example

When developing a model, we need to consider earlier what format is required in production environment, since it may have additional impact on the development of additional transformations, e.g. :



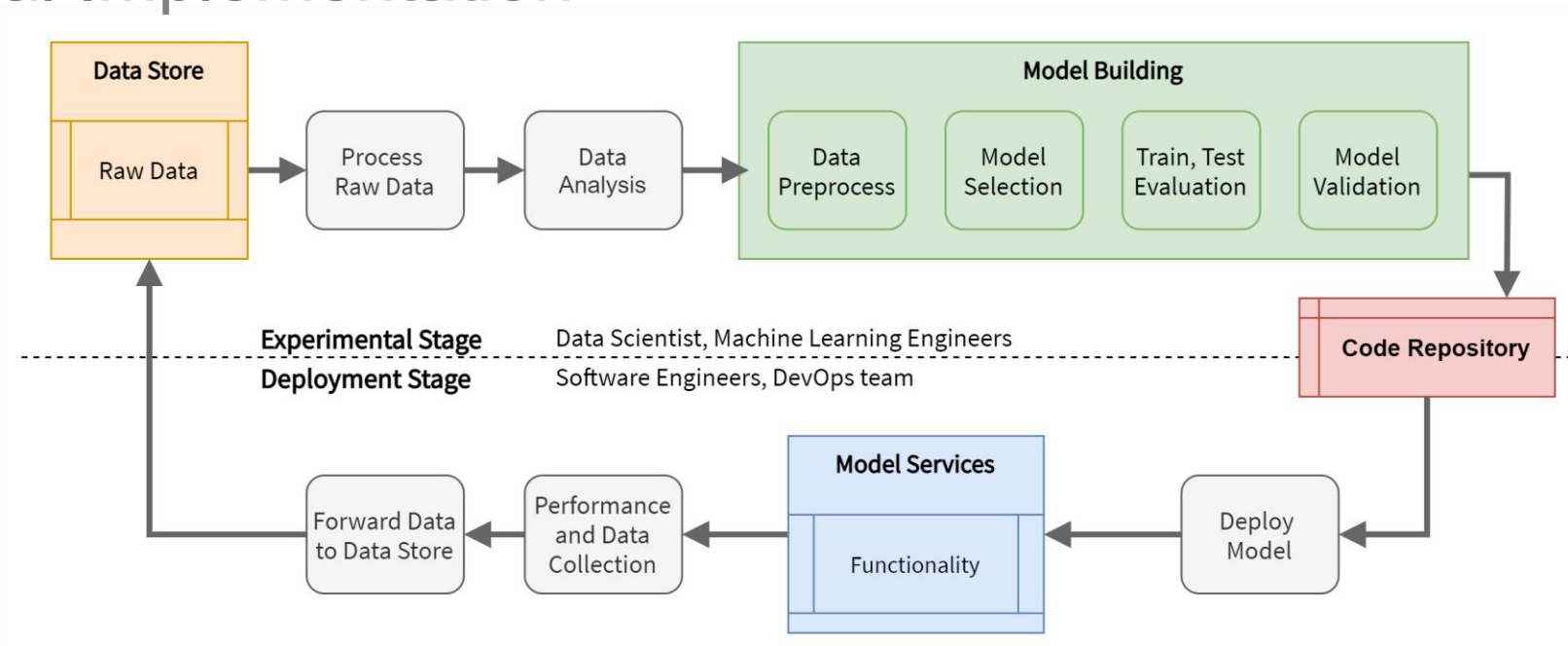
03: Model Deployment Requirements



- The process is divided into distinct subsets of activities, aimed to simplify and standardize the development and distribution of a software. **Each of them constitutes a phase of the process and it can have several levels of automation:**
 1. Manual Implementation.
 2. Continuous Model Delivery.
 3. Continuous Integration / Continuous Delivery of pipeline

03: Model Deployment Requirements

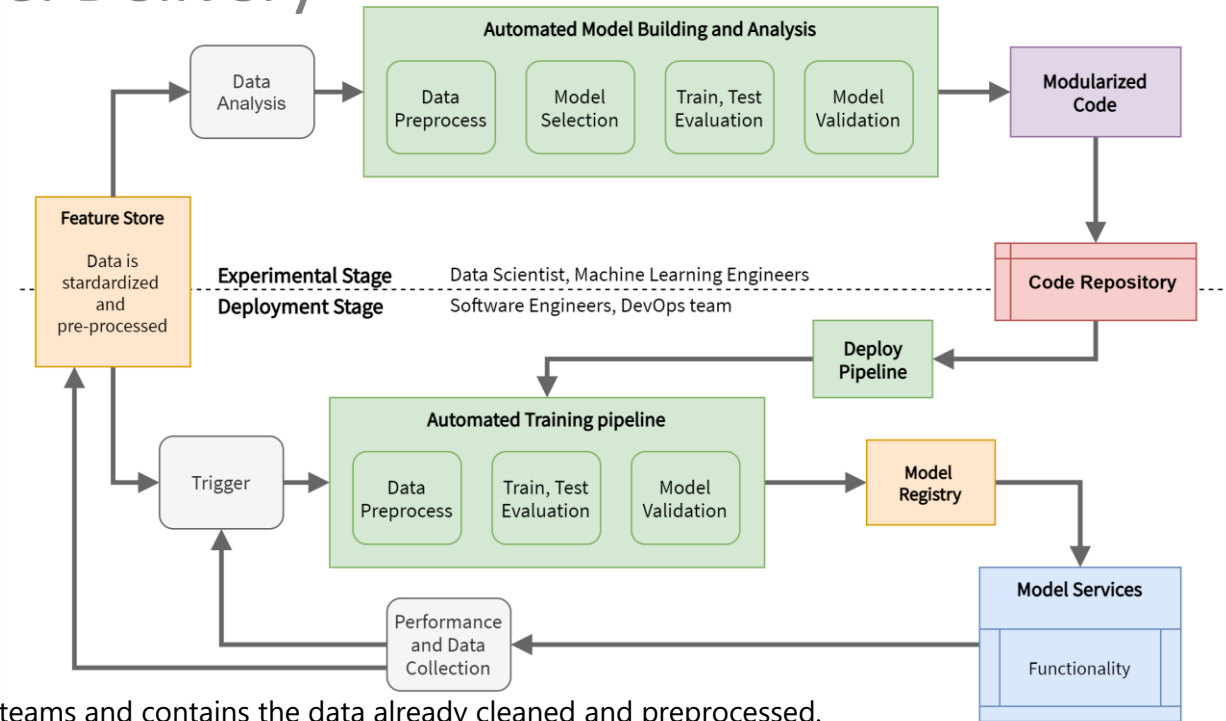
Manual Implementation



- **Manually package the model (jupyter notebooks)** in a structure that can be used and interfaced with other components and put it in a code repository.
- **Merge a Git feature branch** to the main branch.
- Software engineers must take the model from the code repository and manually integrate it into the application and the DevOps team is left with the task of monitoring the application in functional and performance terms.
- The weakest points of this solution are:
 1. The setup **is not resilient to changes** in data and business needs.
 2. Everything **is handled manually**, without any pipeline and automation technique.
 3. Can easily **become "person dependent"**, i.e. if the responsible for a part goes on vacations, the probability of risks and incapacity of solving issues increases.

03: Model Deployment Requirements

Continuous Model Delivery



1.Feature Store -> data accessible to all teams and contains the data already cleaned and preprocessed.

2.Automated Model Building and Analysis, which is an engineering of the model building phase. The output is a Modularized code, that allows you to standardize and automate the next steps.

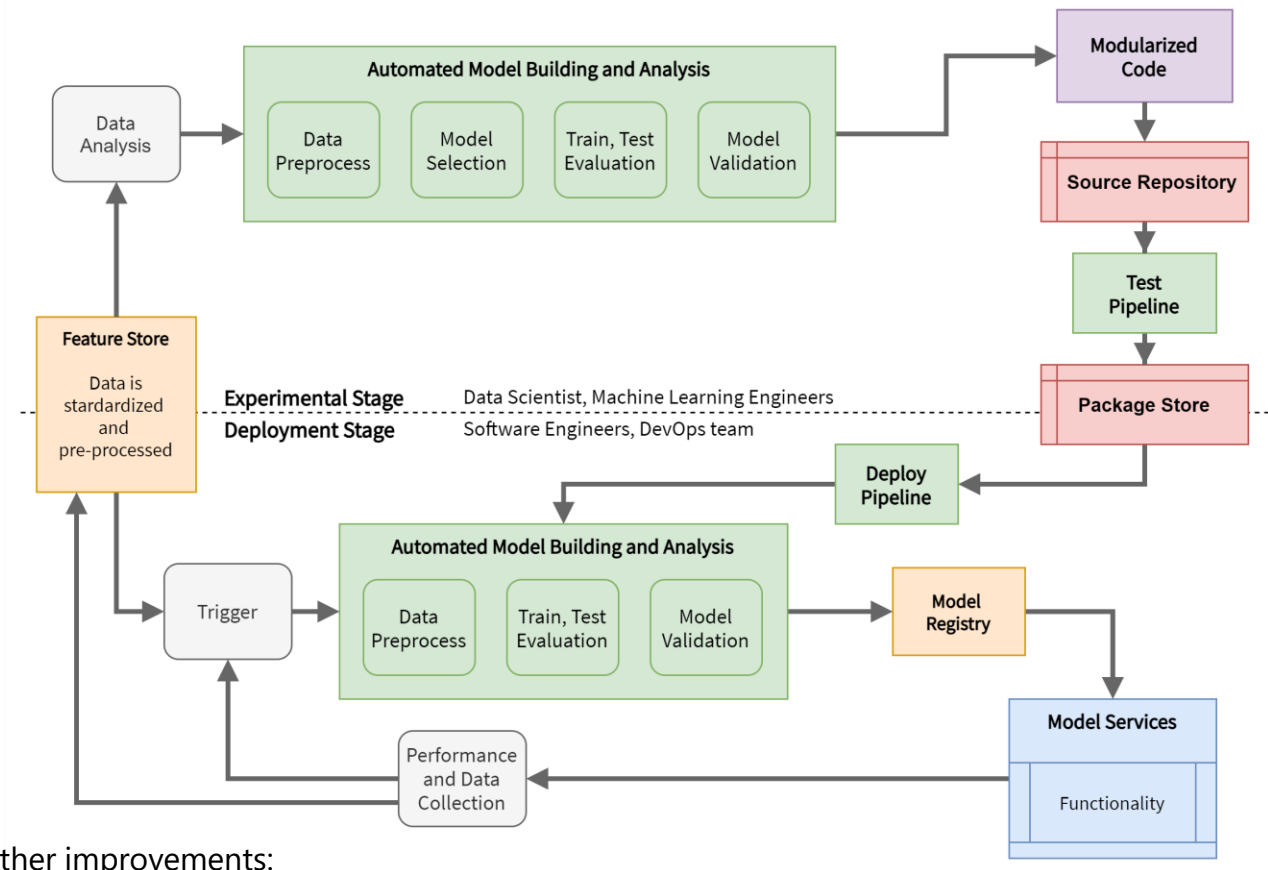
3.The Deploy Pipeline, which is a pipeline designed to put the model into production.

4.The Automated Training pipeline, which is launched from the Deploy Pipeline or a Trigger and interfaces with the Feature Store to get the data that will be used for training. The final model is placed in a Model Registry (**MLflow playground**). The trigger can be enabled manually, or it can be enabled automatically by events such as the change of distribution in the data collected by monitoring the model in production.

5.The Model Registry allows you to keep track of the various models produced and versioning them. Makes life easier when choosing the automatically the best model, by comparing it with some baseline version.

03: Model Deployment Requirements

Continuous integration/Continuous delivery of pipelines



This new setup introduces other improvements:

1. Testing, to be able to automatically test code building pipelines and package only those pipelines that pass the tests. They can also be reused on different domains and with very different neural network architecture.

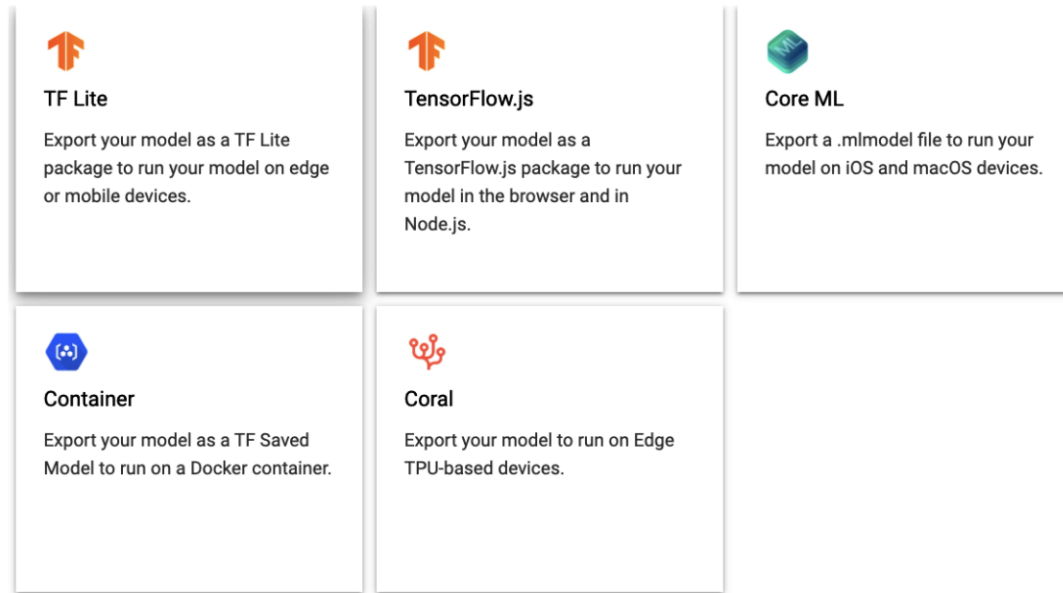
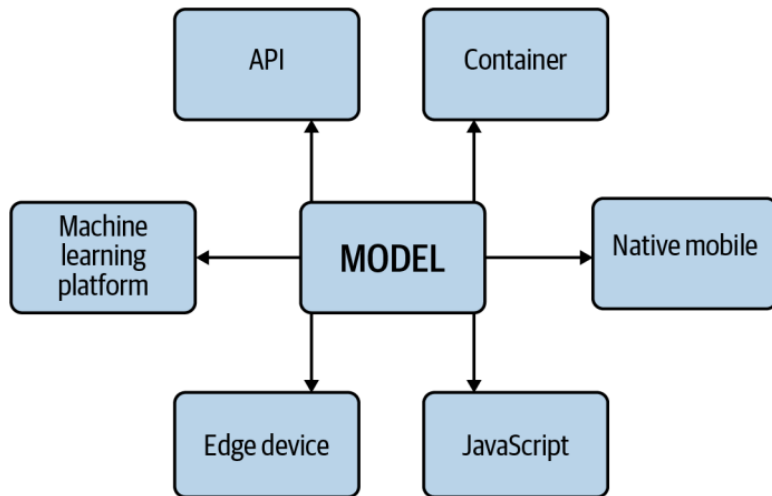
2. The Package Store is a pipeline container, not mandatory. It works in a very similar way to the Model Registry, where all teams can access packaged pipelines ready for deployment.

03: Model Deployment Requirements

Continuous Deploy/Delivery

- If we have several model pipelines that need to be run in a certain order, and they generate different outputs, **the process should be automated and independent from the host environment where the delivery happens.**
- There are several options:

Google Cloud Platform AutoML example



- **Containers will be our focus**, as one of the most versatile solutions for many use cases. They will make models governance easier!

03: Model Deployment Requirements

Containerization

- Environmental compatibility is a headache, in addition to scaling, security, maintenance and upgrade of software and hardware.
- For hosted environments, we must work hard on the DevOps to ensure the environments are the same. Usually, **we do this part using requirement file and locally installing python packages.**

requirements.txt

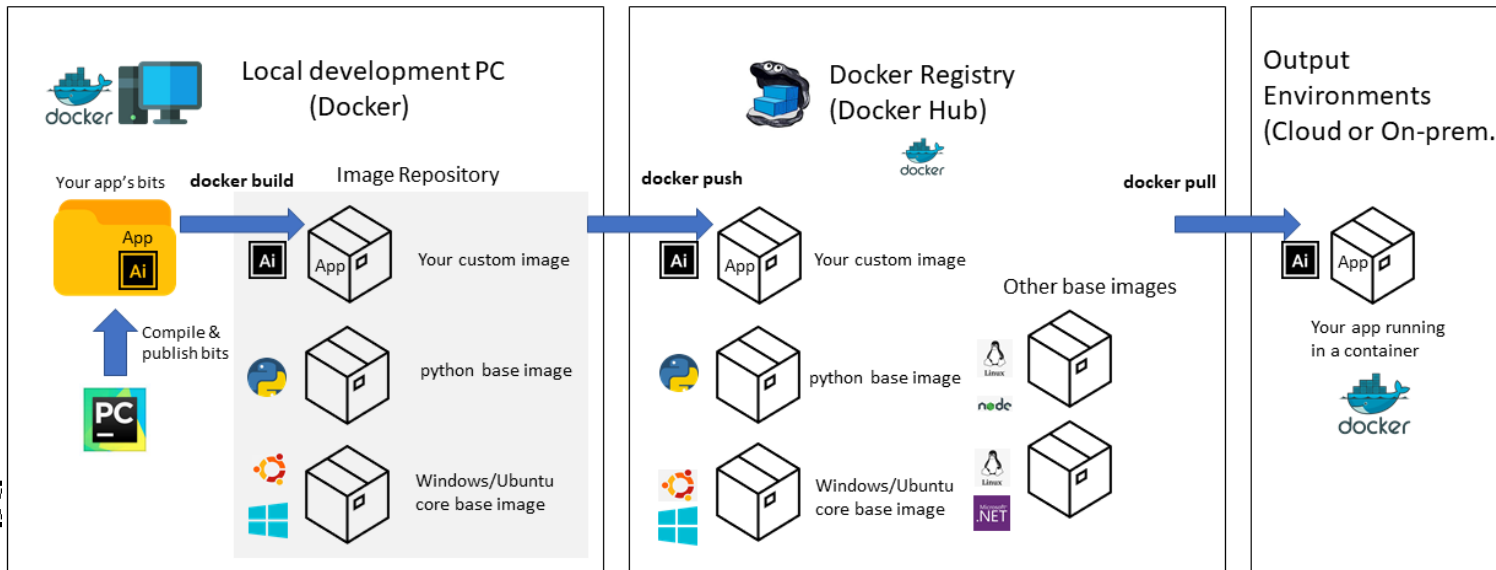
```
1 pandas==1.2.1
2 matplotlib==3.4.0
3
```

install

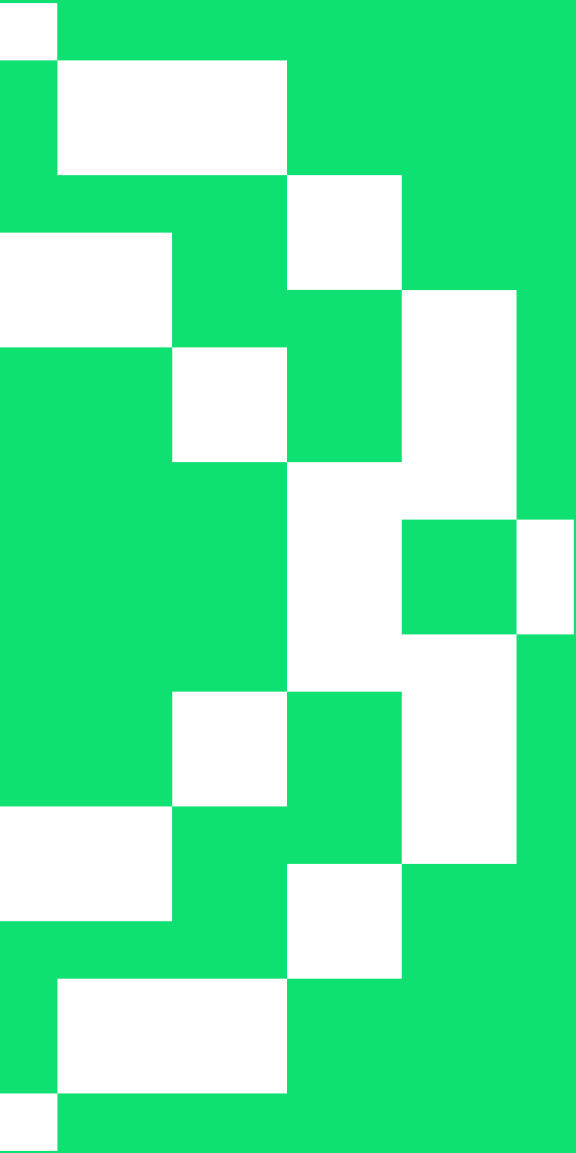
```
1 pip install -r requirements.txt
```



- Containers are tools to produce **reproducible environments by isolating and packaging everything that your program needs**, independent of the target/host environment where you intended to run the program.
- Many tasks can be isolated and served in such a way, including model deployments.



- Because of their architecture, **they are much lighter on resources compared to a full-blown virtual machines VM.**
- Containers with our code/program/model in **it can be distributed to team members and can be publicly released with minimal environment incompatibility issues.**

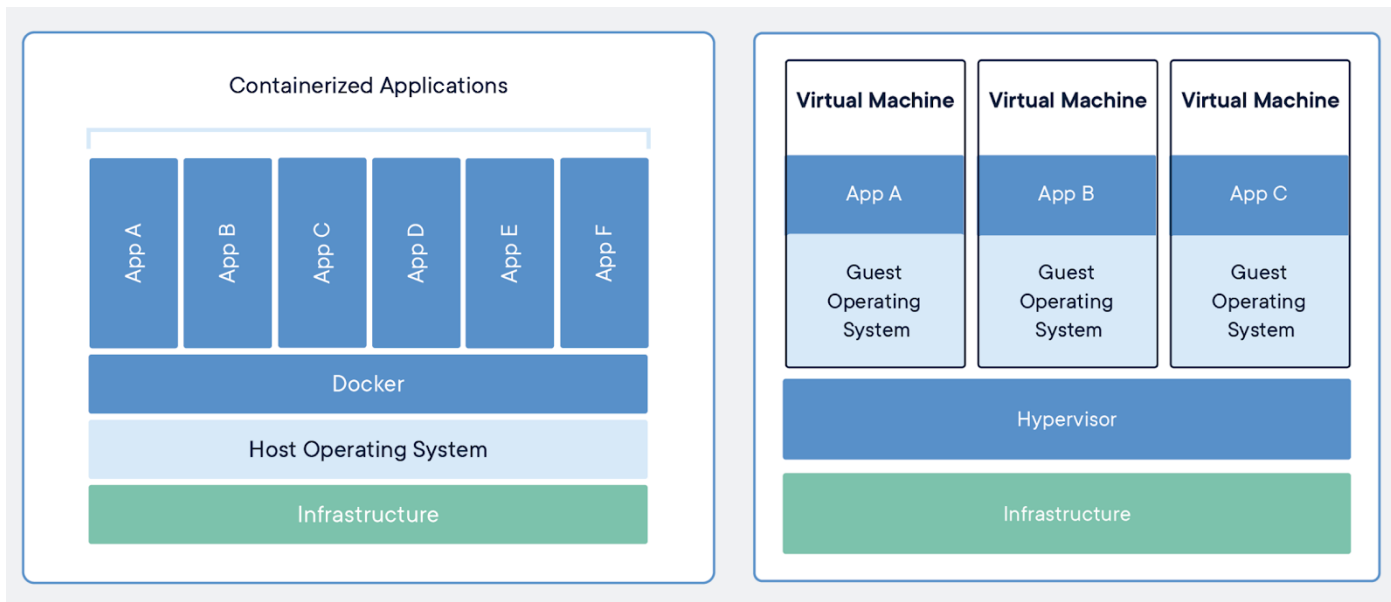


Model Deployment Containers

03: Model Deployment Requirements

Containers

- A container is a **standard unit of software that packages code and all its dependencies**, so the application runs quickly and reliably from one computing environment to another.
- Using containers to separate different use cases makes our life easier to test-run new packages, modules and framework versions, without breaking the entire system. **We do not need to install every tool on a local host.**
- A container was an answer for some "heavy aspects" of the standard virtual machines. Virtual machines are an abstraction of physical hardware, turning one server into many servers. Each VM includes a full copy of an operating system, necessary binaries and libraries.
- **You can think of a container as a virtual machine**, but containers virtualize the operating system instead of hardware, which makes them portable and efficient.
- With a container **we have great flexibility to add data analytics and machine learning tools**, including jupyter tools for the most resistant ones.



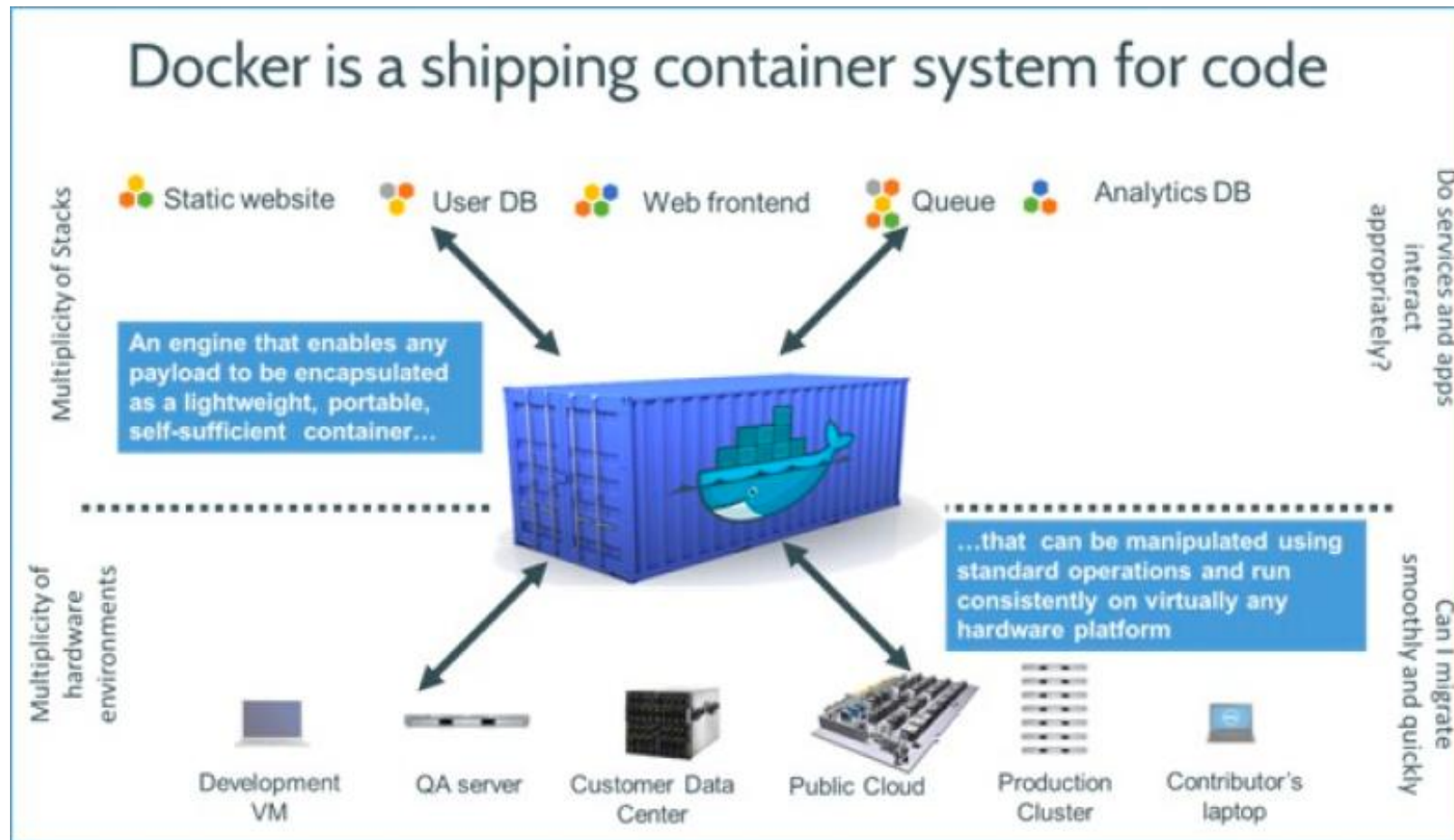
- We can have multiple containers that can run at same machine, and share the OS kernel with other containers, each running as isolated processes in user space.
- Each container is an abstraction at the app layer of each pipeline that we built.
- We don't have any more the risk of having to delete the virtual environment or reinstall the operating system if crashed by bad packages or frameworks.

How we can build a container? **Docker**

03: Model Deployment Requirements

Containers

- Docker is a tool for developing and deploying applications using containers, such as you may bundle your code together with its dependencies into one deployable unit.



03: Model Deployment Requirements

Docker



Key aspects of Docker:

1. Reproducibility:

- Different analysis and different models can be shared with the other collaborators without replicate your whole stack as well. As a Data Scientist or a Client of the service, you will not need to be hours or days setting up the entire environment.

2. Portability:

- Docker enables you to bundle your code and dependencies into containers that can subsequently be ported to various computers, even if the underlying hardware and OS are different from the original machine.

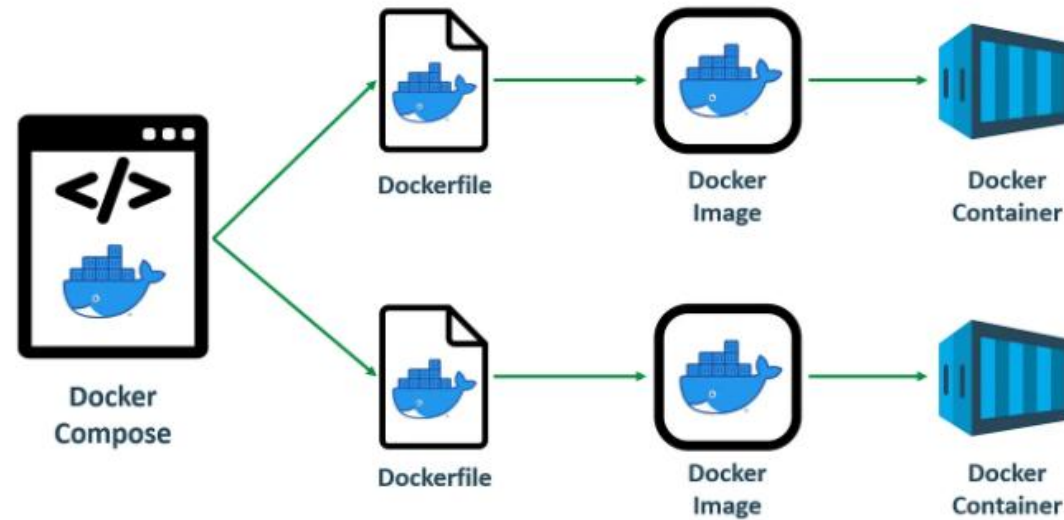
3. Integration:

- Cloud providers and physical servers may be provisioned using Docker Cloud to construct Docker nodes the most relevant players of the markt are connected with Docker Cloud.

Container image versions can be tracked using Docker, as well as who developed them and with what. Versions may be rolled back as well.

03: Model Deployment Requirements

Docker



- **The Dockerfile is at the heart of creating containers.** Anytime you are creating a container, you must have the Dockerfile present in the current directory.
- **Contains all the commands required to generate a specific image** that is read by Docker to build that image automatically.
- **An image is a snapshot of a Docker virtual machine** at a certain time and place.
- **Images are immutable.** It's possible to erase a record after you've produced it, but you can't change it.
- **Experimentations will not affect images** in the new environment. Also it is easier to debug a problem.
- In the container **the image offers all the required files to operate a set of processes.**
- **The applications are isolated from other processes** that share the same kernel, so no one process can affect the work of others.

03: Model Deployment Requirements

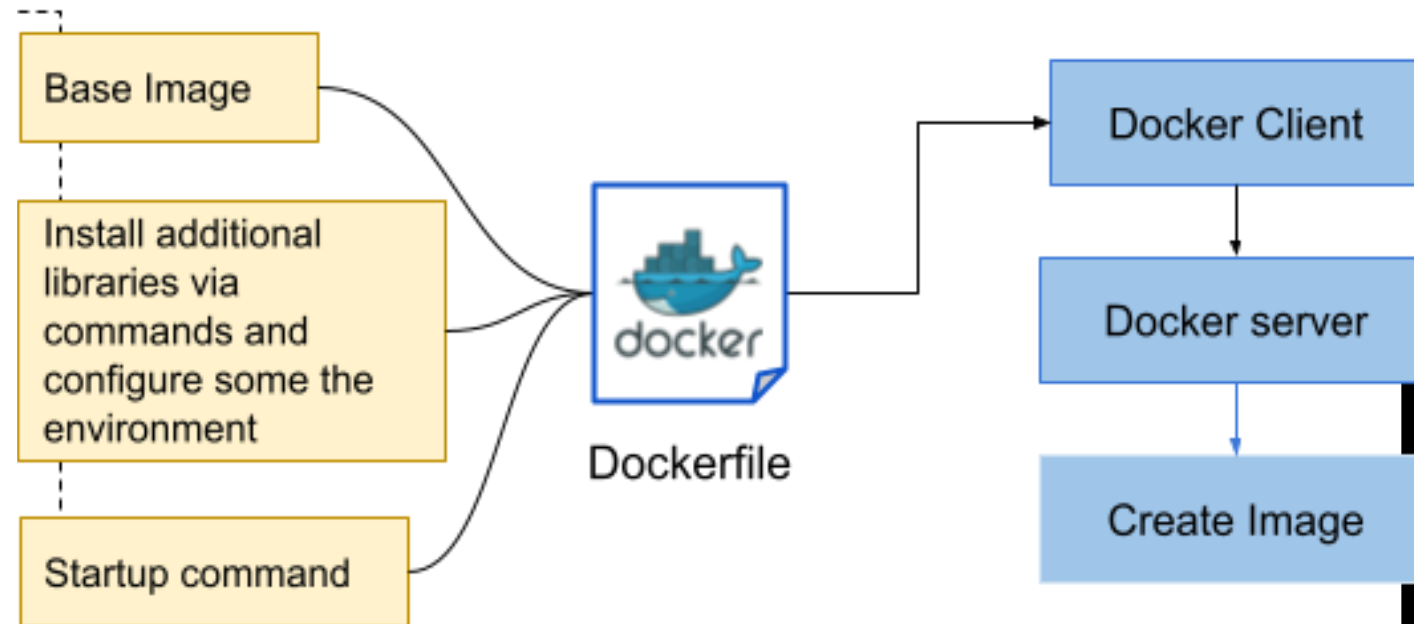
Dockerfile structure

base image is an ubuntu
FROM ubuntu:latest

RUN apt-get install -y python3
-pip python3-dev build-essential

RUN pip3 install -r
requirements.txt

CMD ["python3","main.py"]

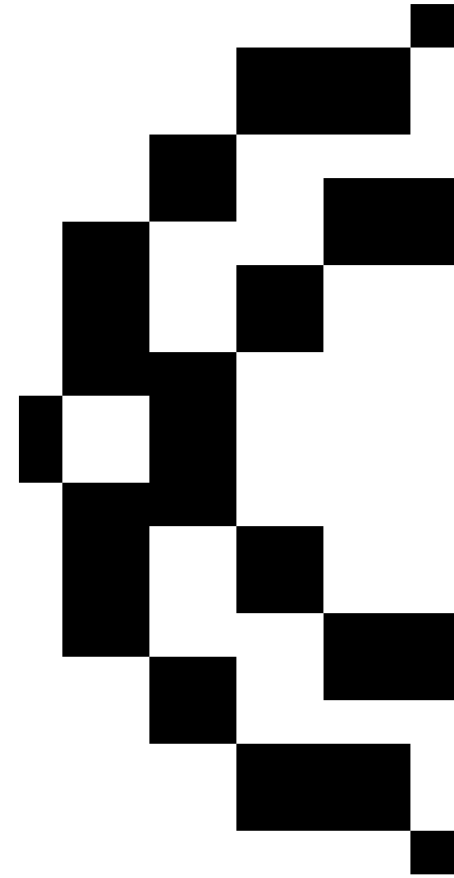


- Example of commands that can use in a dockerfile are:
 - ❑ **FROM** - Creates a layer from the ubuntu
 - ❑ **RUN** - Builds your container
 - ❑ **CMD** - Specifies what command to run within the container

03: Model Deployment Requirements

Dockerfile structure

✓	docker	●
✓	src	●
🐍	serve.py	U
🐍	train.py	U
!	config.yaml	U
🚢	Dockerfile	U
≡	requirements.txt	U



03: Model Deployment Requirements

Dockerfile structure

```
1  FROM python:3.10-slim
2
3  WORKDIR /app
4
5  COPY . .
6
7  RUN pip install --no-cache-dir -r requirements.txt
8
9  EXPOSE 5000
10 CMD ["python", "src/serve.py"]
```

- Pulls the official Python 3.10 “slim” image from Docker Hub - “slim” means it’s a minimal Debian-based image with Python installed, keeping the final image smaller.
- All subsequent commands (COPY, RUN, CMD, etc.) will operate from /app. If /app doesn’t exist, Docker will create it.
- Copies everything from your local directory (where you run docker build) into the container’s /app folder, where the first . refers to the host context, the second . to the container’s WORKDIR.
- Runs pip install inside the container to install all packages listed in requirements.txt, --no-cache-dir tells pip not to save download caches, reducing image size.
- Indicates that the containerized app listens on port 5000. When you start the container (with docker run), this is the command that will execute by default.

03: Model Deployment Requirements

Problems from past classes

Remember that some tools do not evolve simultaneously, and they depend on different versions of the same packages:

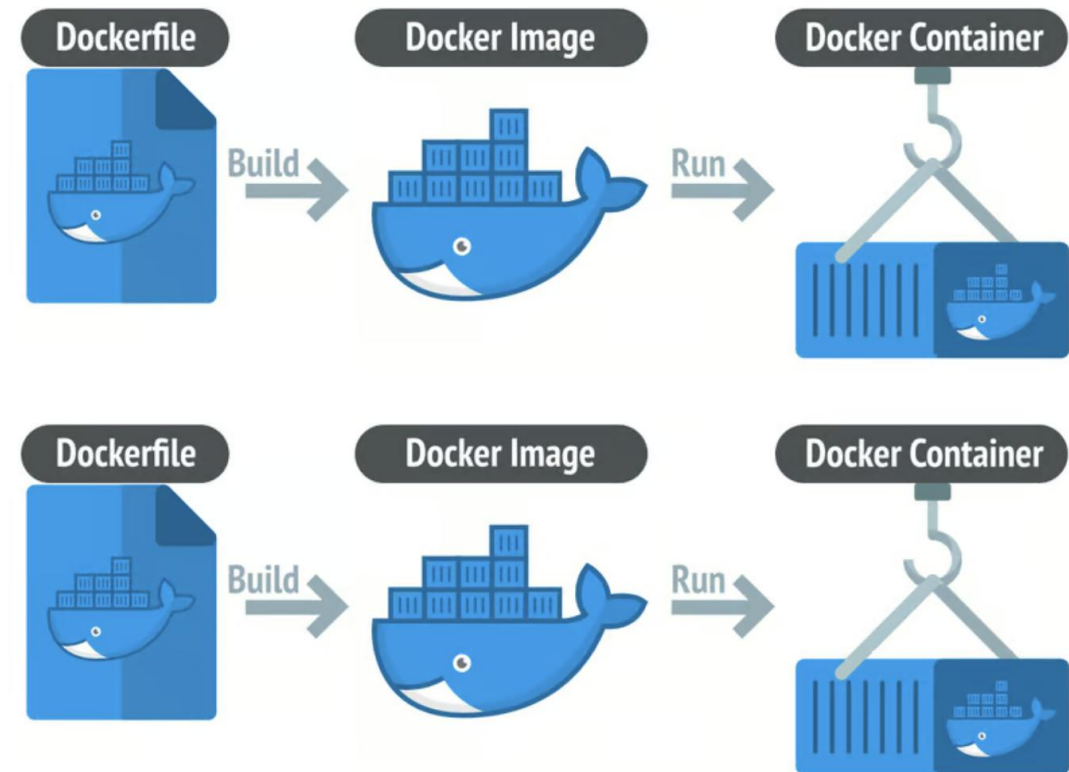
- **Great Expectations** most recent version enters in conflict with Hopsworks.
- **MLflow** issues with Sklearn versions and Setuptools.

Container A:

- Application to test data using GX and with the corresponding requirements.

Container B:

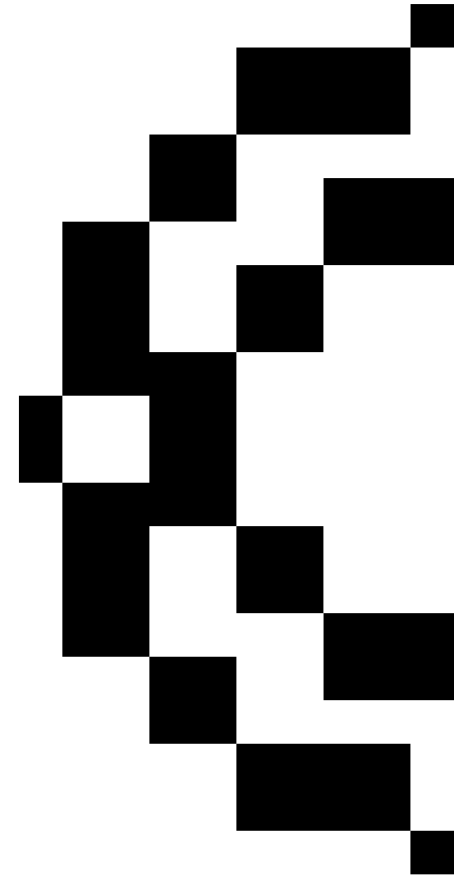
- Application to model experimentation using MLflow and with the corresponding requirements.



03: Model Deployment Requirements

Dockerfile relevant commands

Command	Description
FROM	Base image from which new image will be created
MAINTAINER	Metadata about who owns this
LABEL	Using label you can organize images in a proper way
WORKDIR	Sets where CMD is executed from
RUN	Executive directive used to build an image
COPY	Lets you copy content to the image
CMD	Provides defaults arguments to the ENTRYPOINT
ENTRYPOINT	Defined when using the container as an executable
ENV	Used to set environment variable
VOLUME	Used to creat a mount point
EXPOSE	Exposes port(s) on which container listens for connection
USER	Sets the UID to run the next command as



03: Model Deployment Requirements

.dockerignore

- Before building accordingly to the instructions, **it looks for a file named .dockerignore in the root directory of the context.**
- If this file exists, **it modifies the context to exclude files and directories that match patterns in it. This helps to avoid unnecessarily sending large or sensitive files and directories to the daemon and potentially adding them to images using ADD or COPY.**
- Security issues to create this file: important files such as passwords, secret keys, .git folders contain a lot of information about the project, and you want to avoid to expose the details.
- When you write the Dockerfile, **it's a general practice to use the COPY instruction to copy the files and folders inside the Docker build context.** Each statement inside the Dockerfile results in building a new intermediate image layers -> cache invalidations and wastage of resources.

.dockerignore file example

```
# comment
*/temp*
*/*/temp*
temp?
```

Actions

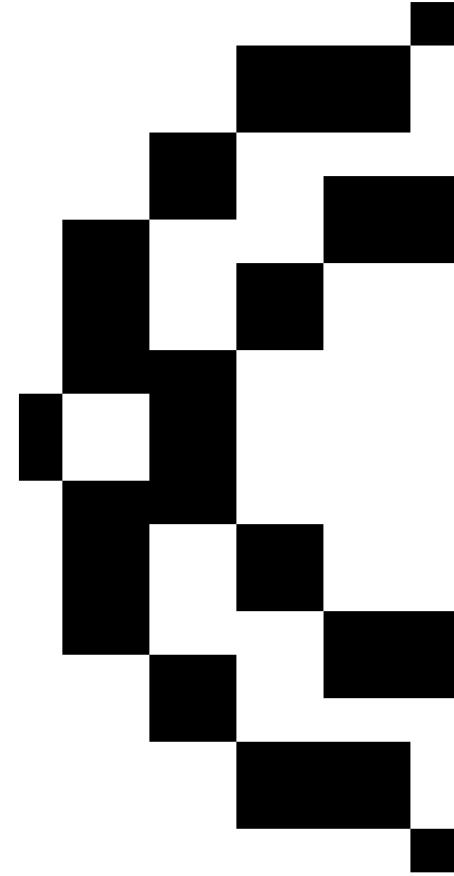


Rule	Behaviour
# comment	Ignored.
/temp	Exclude files and directories whose names start with temp in any immediate subdirectory of the root. For example, the plain file /somedir/temporary.txt is excluded, as is the directory /somedir/temp.
//temp*	Exclude files and directories starting with temp from any subdirectory that is two levels below the root. For example, /somedir/subdir/temporary.txt is excluded.
temp?	Exclude files and directories in the root directory whose names are a one-character extension of temp. For example, /tempa and /tempb are excluded.

03: Model Deployment Requirements

.dockerignore relevant commands

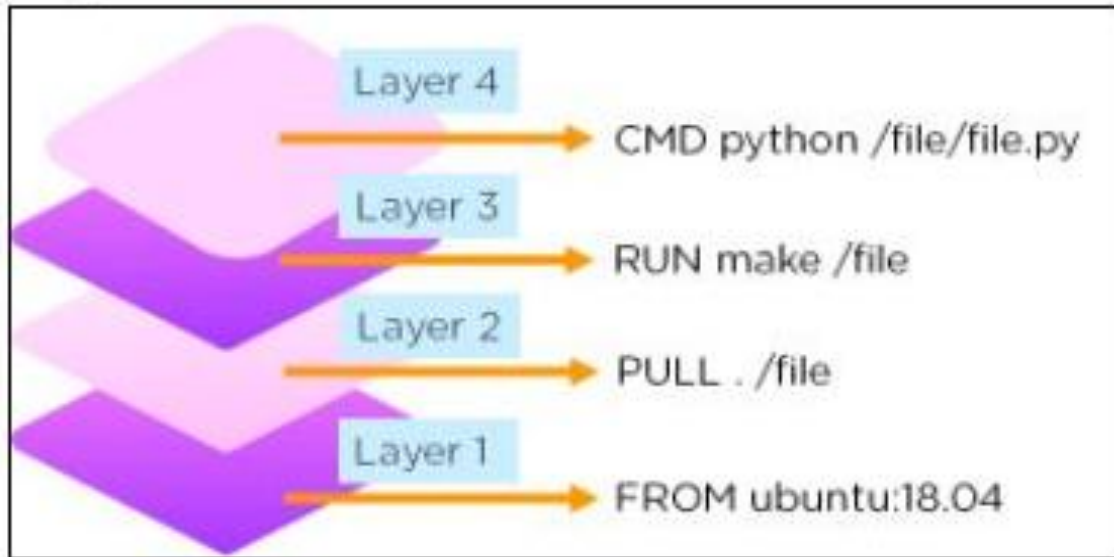
Command	Description
#	Comment
*	One/more character wildcard
?	One character wildcard
[]	Range of characters
!	Exception (not)



03: Model Deployment Requirements

Docker good practices

- There are a series of good practices to implement each relevant component of Docker:
 - **Use Docker official images.** If you build containers on top of existent images, use trusted providers. Check the image source https://hub.docker.com/_/python.
 - **Use .dockerignore to prevent undesirable files and directories** from being contained in the image when we run the docker build command.
 - Take advantage of Docker caches, the layers. Layers are used to build Docker images.



- When building an image, Docker steps through the instructions in your Dockerfile, executing each in the order specified.
- As each instruction is examined, Docker looks for an existing image in its cache that it can reuse, rather than creating a new, duplicate image.
- If you have several layers, then you can order them from the less frequently changed to the more frequently changed.

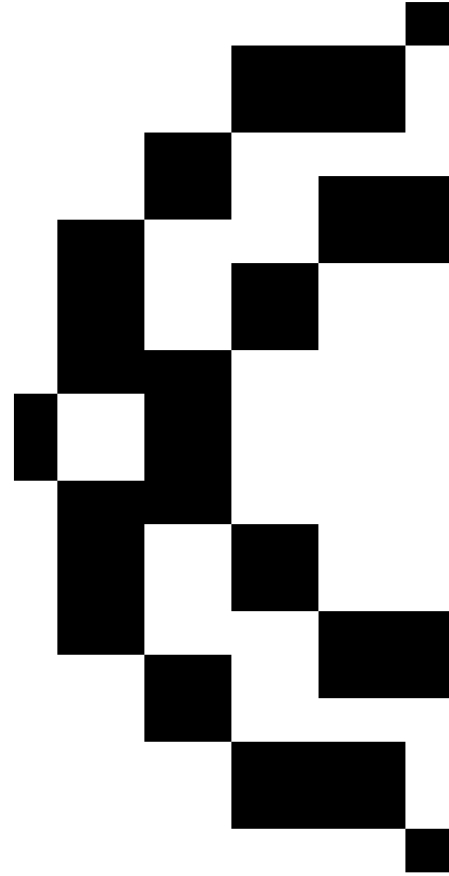
03: Model Deployment Requirements

Docker good practices

- There are a series of good practices to implement each relevant component of Docker:
 - Use a container for each process, which makes the debugging and orchestration far more easier. **Also a container is meant to have the same lifetime as the app it contains.**

```
# train/Dockerfile
FROM python:3.10-slim
WORKDIR /app
COPY train.py requirements.txt ./
RUN pip install --no-cache-dir -r requirements.txt
CMD ["python", "train.py"]
```

```
# serve/Dockerfile
FROM python:3.10-slim
WORKDIR /app
COPY serve.py requirements.txt model.pkl ./
RUN pip install --no-cache-dir -r requirements.txt
CMD ["python", "serve.py"]
```



03: Model Deployment Requirements

Docker good practices

- There are a series of good practices to implement each relevant component of Docker:
 - Use a container for each process, which makes the debugging and orchestration far more easier. **Also a container is meant to have the same lifetime as the app it contains.**
 - **A separate container for each service allows you to grow one of your web servers horizontally** to accommodate the extra traffic.

Docker Compose is a tool that lets you define and run multi-container Docker applications with a simple YAML file

```
version: "3.8"
services:
  api:
    image: myregistry.com/ml-api:1.2.0
    ports:
      - "5000:5000"
    deploy:
      replicas: 4    #means running 4 instances of the same containerized service.

  redis:
    image: redis:6.2-alpine

  store:
    image: myregistry.com/feature-store:2.0.1
```

03: Model Deployment Requirements

Docker good practices

- There are a series of good practices to implement each relevant component of Docker:
 - Use a container for each process, which makes the debugging and orchestration far more easier. **Also a container is meant to have the same lifetime as the app it contains.**
 - **A separate container for each service allows you to grow one of your web servers horizontally** to accommodate the extra traffic.
 - There are tags that you can use for the basic image of the container. **Avoid utilizing these tags for deployment containers since these tags will get changed often**, and it might lead to discrepancies in the production environment, e.g. `image: example.com/ml-api:latest`
 - **Use unique tags for deployments.** You can extend your production cluster to numerous nodes with ease. It prevents inconsistencies, and hosts will not fetch any other docker image version.

REPOSITORY	TAG	IMAGE ID
myapp	debug	26eef669f4af
testbuild	latest	26eef669f4af
ubuntu	16.04	005d2078bdfa
ubuntu	latest	1d622ef86b13

One can have repos with same name but different tags

03: Model Deployment Requirements

Containerization

What if ...

- When a model is deployed in several copies, how can the workload be balanced?
- What happens if the model becomes unresponsive, for example, if the machine hosting it fails? How can that be detected and a container reprovisioned?
- How can a model running on multiple machines be upgraded, with assurances that old and new versions are switched on and off, and that the load balancer is updated with a correct sequence?

Solution



- Kubernetes, open source platform for container orchestration.
- No code for steps to set up, monitor, upgrand and stop. Just an API and a configurarion file where the users specify the desired state.
- Easy scalability: models are basically formulas that can be replicated in new machines, and Kubernetes handles its balancing and sincronization framework.

03: Model Deployment Requirements

Kubernetes

- Kubernetes is an **open-source system for automating deployment, scaling, and management of containerized applications**. <https://kubernetes.io/docs/home/>
- It was open-sourced by Google (named Borg internally before going out).
- Kubernetes is a **central orchestration system in large complex software systems**.
- It coordinates a collection or cluster of computers with the purpose of working as a single unit.
- When we orchestrate containers with it, we are agnostic to which computer runs the container.
- Kubernetes takes care of scheduling and distribution of containers to these *unnamed* computers behind the scenes.



Kubernetes can be an "overkill" solution if our problem is not so complex. Its usage should be the result of a deliberate cost benefit analysis in comparison to other hosted solutions or managed solutions.



Model Deployment

Kubernetes

03: Model Deployment Requirements

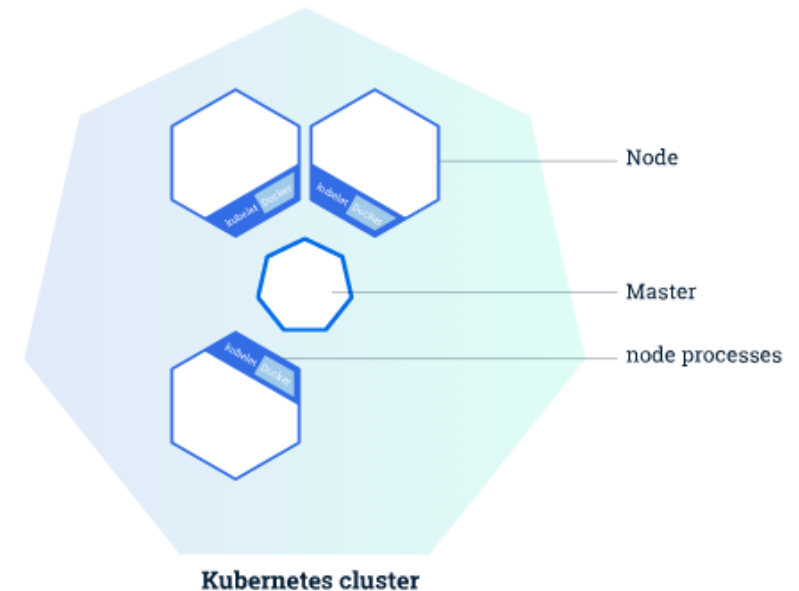
Kubernetes

A Kubernetes cluster has two resources:

- **Master:** coordinator of the cluster.
- **Nodes:** workers that are essentially VMs or computers that run containers.

Relevant concepts:

- **kubelet:** is an agent or program which runs on each node, being responsible for the communication between the Kubernetes control plane and the nodes where the actual workload runs.
- **kubect:** is the command line tool for communicating with a Kubernetes cluster's control plane, using the Kubernetes API,
kubectl [command] [TYPE] [NAME] [flags]



Each node runs kubelet that manages it and communicated with the master. The deploy sequence essentially involves the following:

- kubectl tells the master to start application containers.
- Master schedules containers to run.

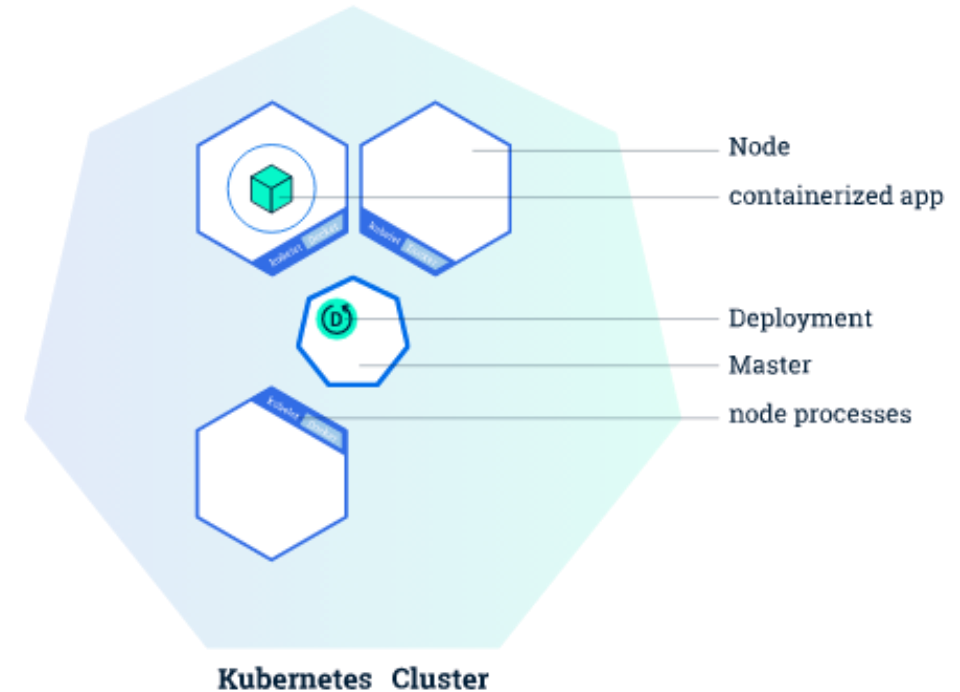
03: Model Deployment Requirements

Kubernetes

- Once we have a cluster running, we can deploy containerized applications (one or many) using a certain configuration.
- This configuration is a set of instructions to Kubernetes to set up the application on the cluster**, where we map each container to the nodes.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: notes-app-deployment
  labels:
    app: notes-app
spec:
  replicas: 2
  selector:
    matchLabels:
      app: notes-app
  template:
    metadata:
      labels:
        app: notes-app
    spec:
      containers:
      - name: notes-app-deployment
        image: pavansa/notes-app
        resources:
          requests:
            cpu: "100m"
        imagePullPolicy: IfNotPresent
        ports:
        - containerPort: 3000
```

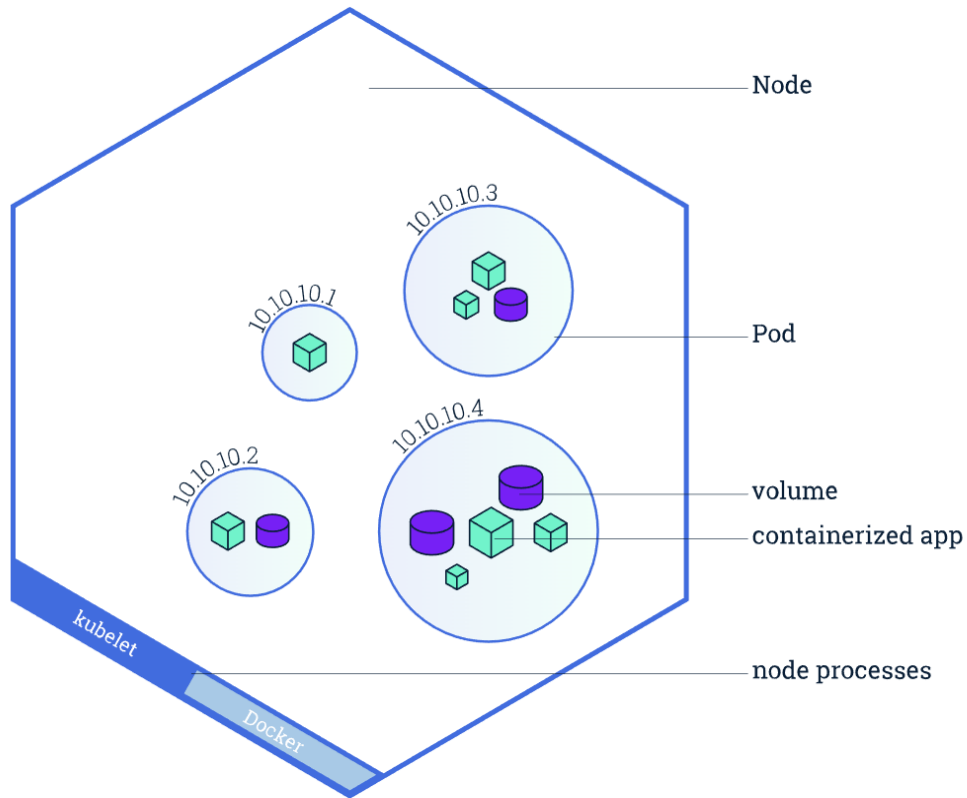
yaml file to
deploy



- When creating a deployment, **we need to specify container images as well as the number of copies of each image we want to instantiate.**
- kubectl that uses an API to interact with the Master, will be responsible for the deployment.
- Replicas are clones that facilitate self-healing for pods.** As with most processes and services, pods are liable to failure, errors, evictions, and deletion.

03: Model Deployment Requirements

Kubernetes



When an app is deployed as a container, it is encapsulated in a pod on a specific node. **Pods are the most basic unit in Kubernetes.** It is pods that are created and destroyed, not individual containers. One node can have several pods.

A pod is a collection of containers that share structures and can be related between each other:

- ☐ **Storage** (volumes)
- ☐ **Networking and IP address**

These related containers can be a web server and a database for instance.

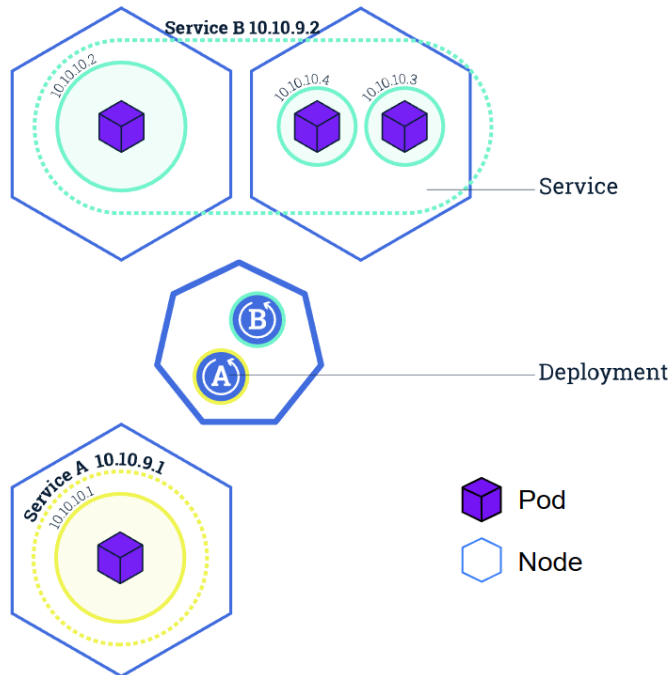
We can also have a single container in a pod.

03: Model Deployment Requirements

Kubernetes

To deploy your application, you need to expose the application to the outside world!

- A Kubernetes Service is an abstraction layer which defines a logical set of Pods and enables external traffic exposure, load balancing and service discovery for those pods.
- A Service is defined using YAML or JSON and lets pods get deleted and replicated in the cluster with no influence on our app. Couplings between pods.
- The pods with IPs cannot be typically accessed from the outside. Instead, a service can be used to allow external connections.



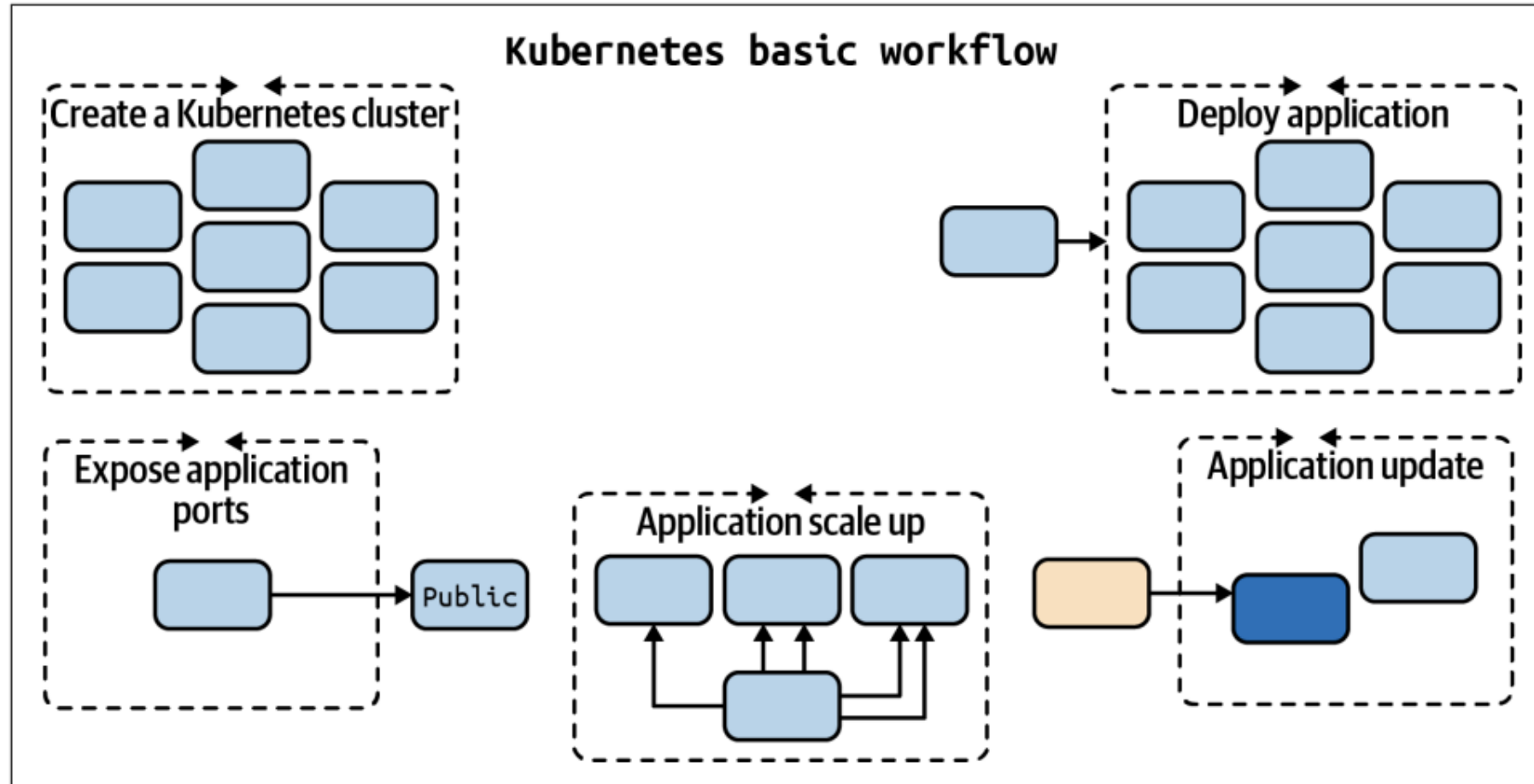
yaml file for service

```
apiVersion: v1
# Indicates this as a service
kind: Service
metadata:
  # Service name
  name: notes-app-deployment
spec:
  selector:
    # Selector for Pods
    app: notes-app
  ports:
    # Port Map
    - port: 80
      targetPort: 3000
      protocol: TCP
      type: LoadBalancer
```

Service Type	Use Case	Accessibility	Resource Allocation
ClusterIP	Internal communication between application components	Within the cluster only	Minimal resources needed
NodePort	External accessibility for web applications or APIs	Accessible from outside the cluster via a high-numbered port on the node	Additional resources needed
LoadBalancer	Production environments with high traffic volumes	Accessible from outside the cluster via a load balancer	Significant resources needed
Cloud Provider's Load Balancer	Using a cloud provider for Kubernetes	Accessible from outside the cluster via the cloud provider's load balancer	May result in cost savings and better performance

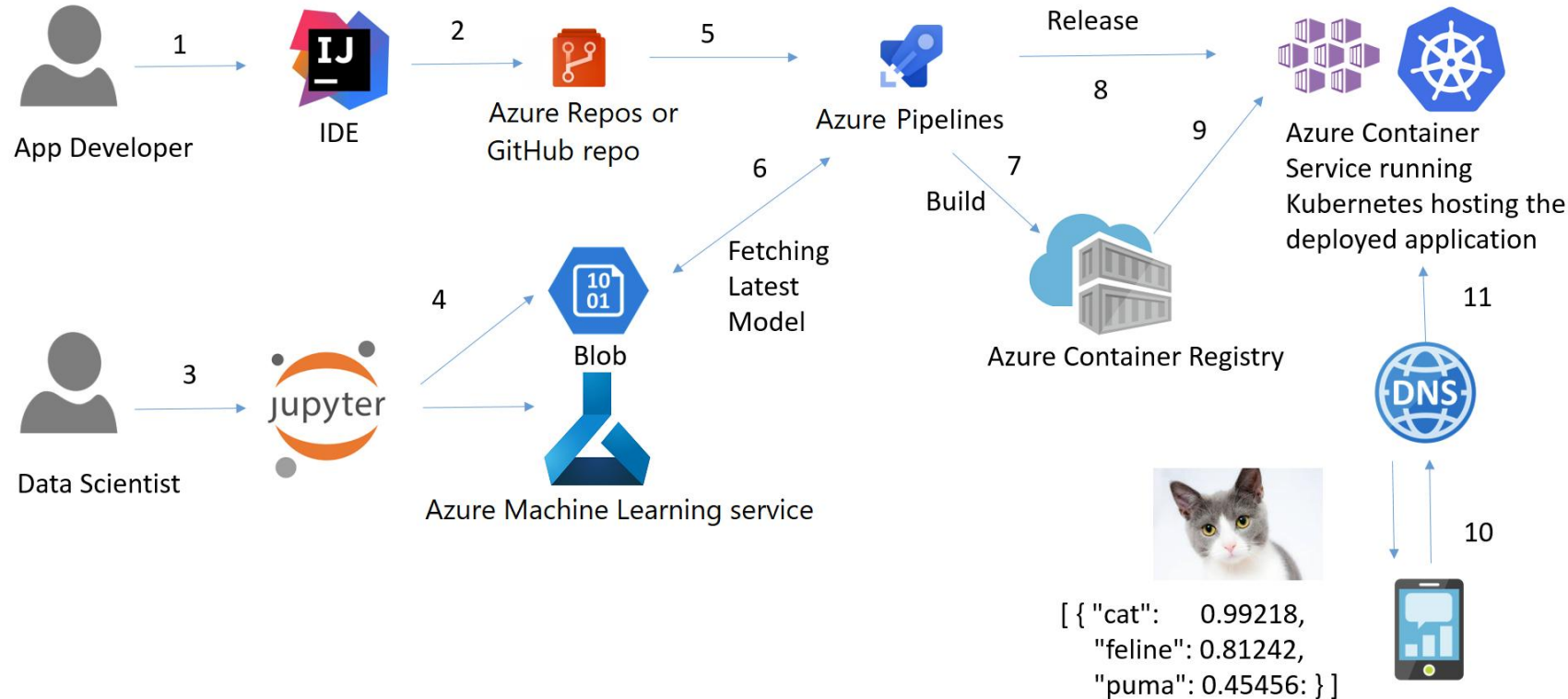
03: Model Deployment Requirements

kubernetes final resume



03: Model Deployment Requirements

Final picture



Model Serving: The software that wraps the model to answer queries (e.g., FastAPI, MLflow). Next week.

Model Deployment: The infrastructure that hosts the served model, manages traffic, and ensures uptime (e.g., Kubernetes, Azure, Load Balancers, Docker).

03: Model Deployment Requirements

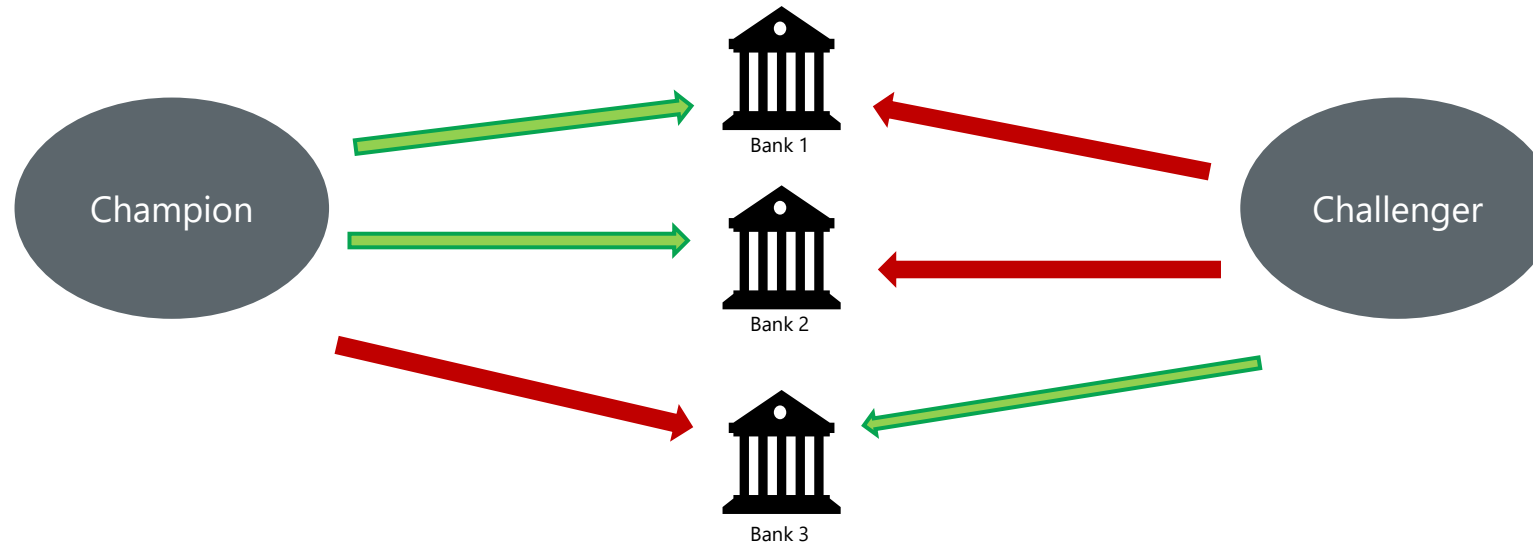
Deployment Architectures: Real-Time vs. Batch

	Real-Time (Online) Inference	Batch (Offline) Inference
What is it?	Predicting one observation at a time, instantly.	Predicting millions of observations all at once.
The Trigger	A user action (e.g., clicking "checkout").	A schedule (e.g., 2:00 AM every Sunday).
Latency Requirement	Milliseconds (Speed is critical).	Hours (Throughput is critical).
Infrastructure Focus	High availability, APIs (FastAPI), Load Balancers.	Heavy compute, parallel processing, big data storage.
Banking Example	Credit Card Fraud Detection (Must block the card <i>now</i>).	Monthly Churn Prediction (Scoring all customers overnight).

03: Model Deployment Requirements

Final remarks on model deployment

- When sending a new model version to production, we want to avoid downtime for real time scoring. **The basic idea is that a new system can be set up next to the stable one, and when it's functional, the workload can be directed to the newly deployed version deployment** (Kubernetes handles this natively).
- Another more advanced solution to mitigate the risk is **to have canary releases (also called canary deployments)**. The idea is that the stable version of the model is kept in production, but **a certain percentage of the workload is redirected to the new model, and results are monitored**.



Canary deployments happen in production to limit the 'blast radius' of potential failures. If a model rolled out to a small group of banks encounters an issue, the rest of the system is protected.